



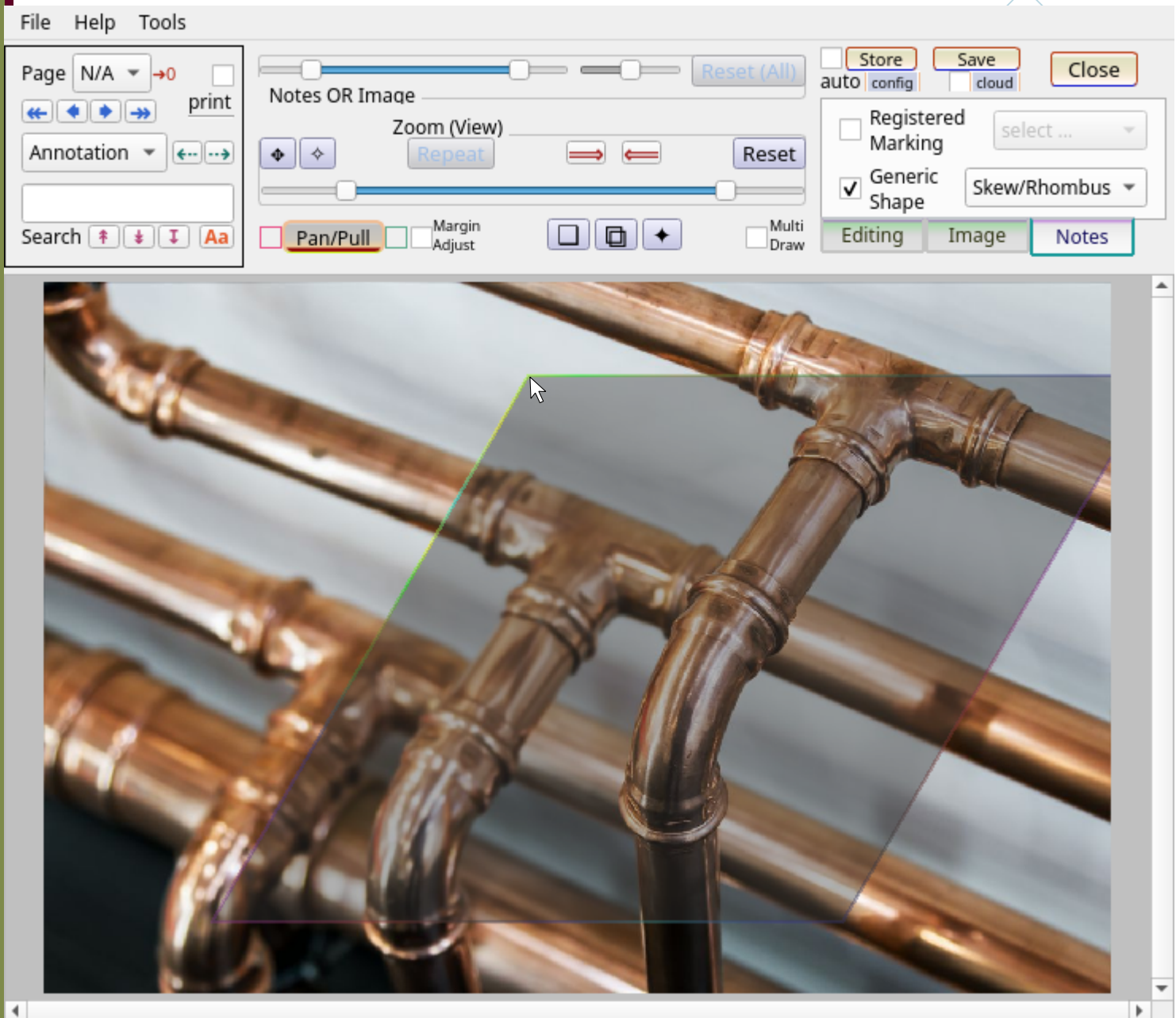
ConceptsDB Database Engine for Automated Analysis of Large-Scale Image Collections

ConceptsDB introduces new representation formats for images, data collections (e.g., image feature vectors) and analytic workflows (e.g., image-processing pipelines) to facilitate the automated analysis of large-scale asset collections, such as an image series.

By systematically combining database engineering with image-processing and similar workflows, ConceptsDB helps developers to implement workflows that may be automatically applied to many images/assets in a series, and repeated (as needed) with different initial parameters.

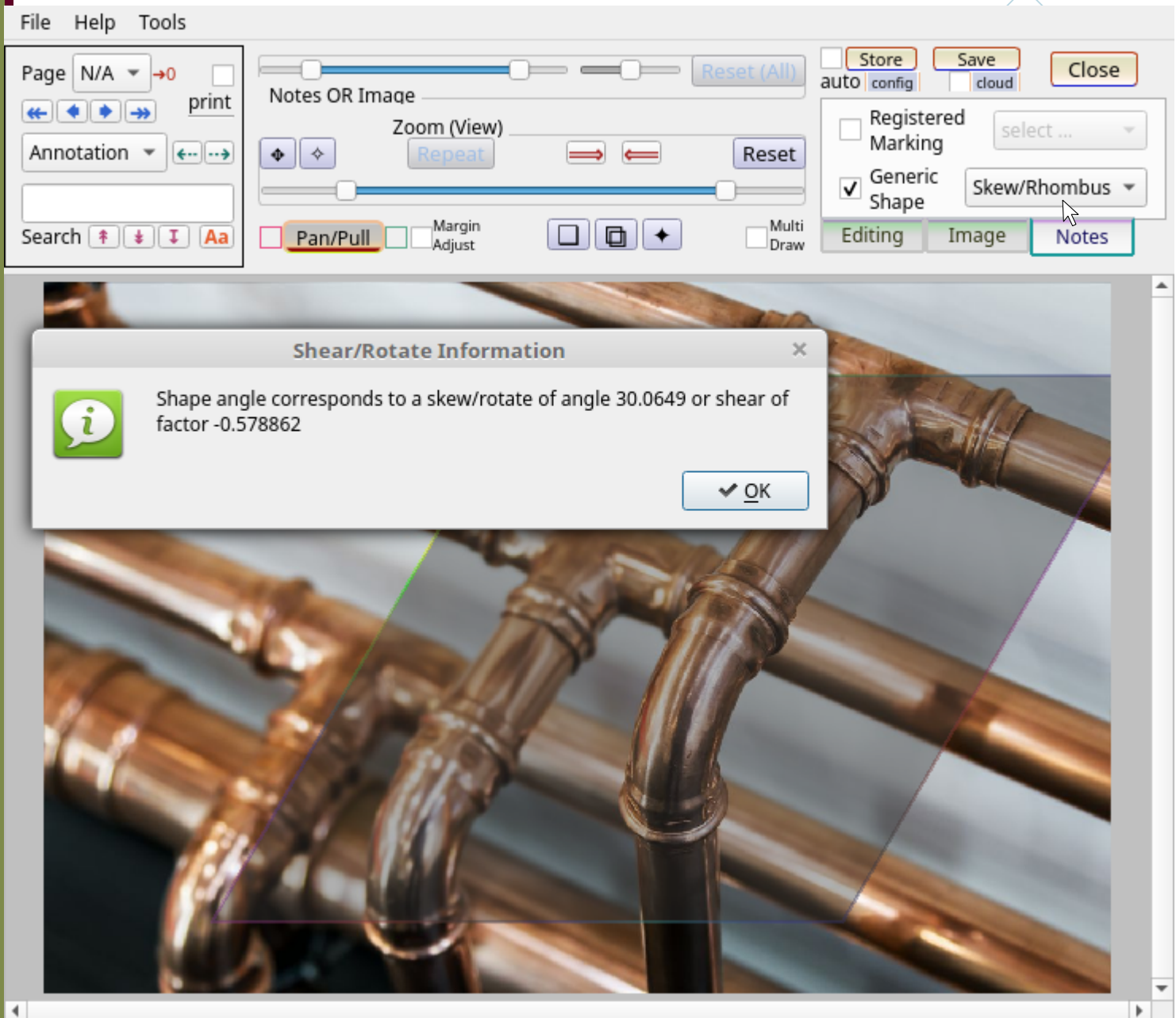
The following slides provide examples of how users can define/refine image-processing pipelines via image and data formats such as:

- The "XCSD" image format — which is a new representation optimized for multi-scale analysis
- A suite of internally-structured annotation styles, as well as GUI event-handlers that allow users to construct these annotations interactively on an image-viewer portal
- GUI components and an event/signal system which developers can use to include XCSD images in their own applications
- Metadata for describing logical inter-relationships between images in the same series, such as lateral relations (pictures whose empirical contents are left/right or above/below one another) or navigable links between images (as when one picture or PDF diagram displays/iconifies a region seen as well, in greater detail, in a connected image)
- Tools for implementing and styling cross-image navigation through GUI components and user cues



Example image-analysis pipeline: slides 1-18

This pipeline begins with establishing parameters for a skew-transform via an annotation-based measurement (screenshot on this slide)



Special annotation classes for measurements such as skew/rotate angles and shear factors (slides 1-2)

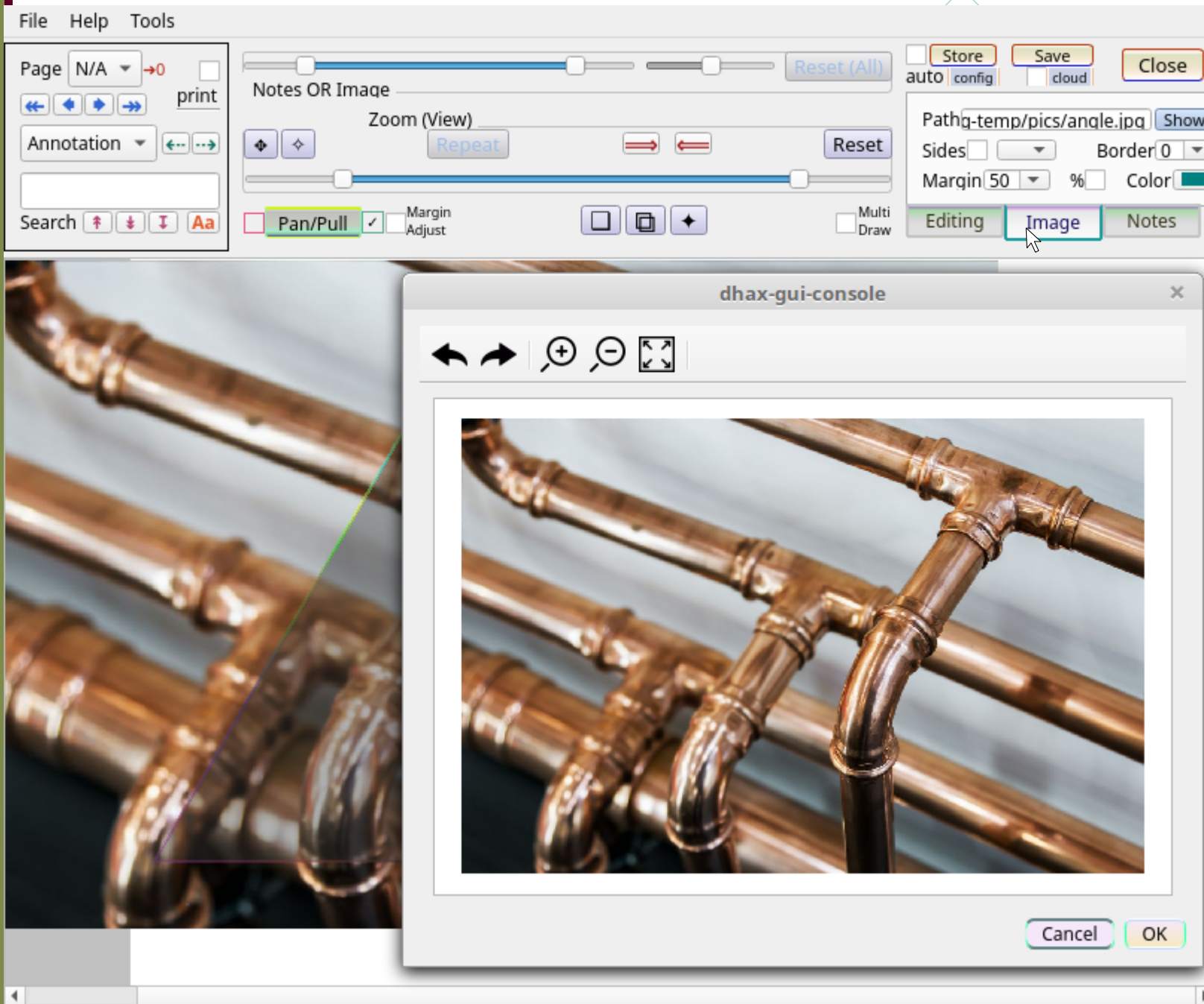
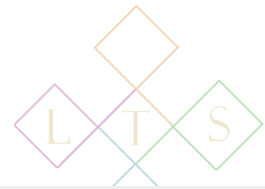
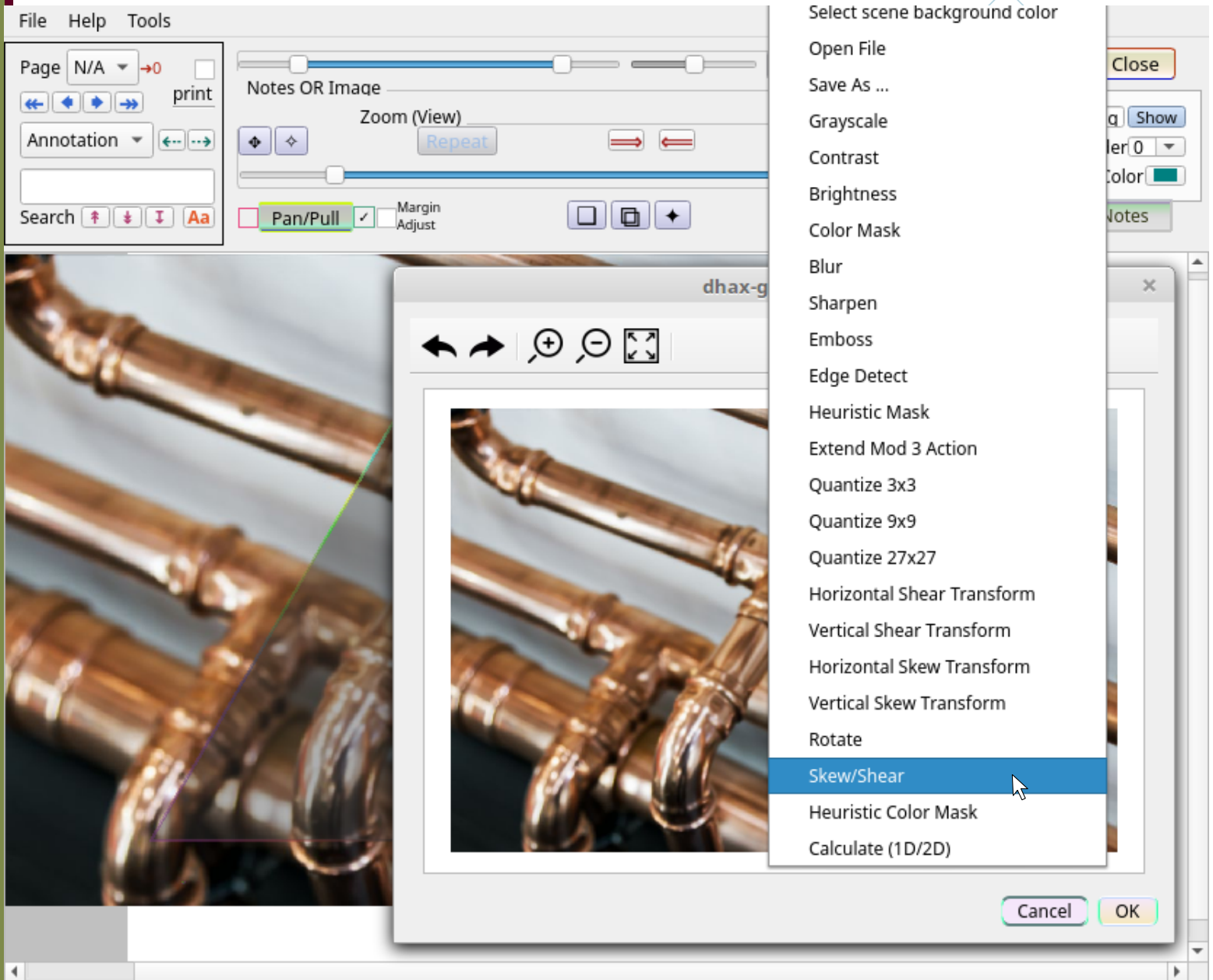
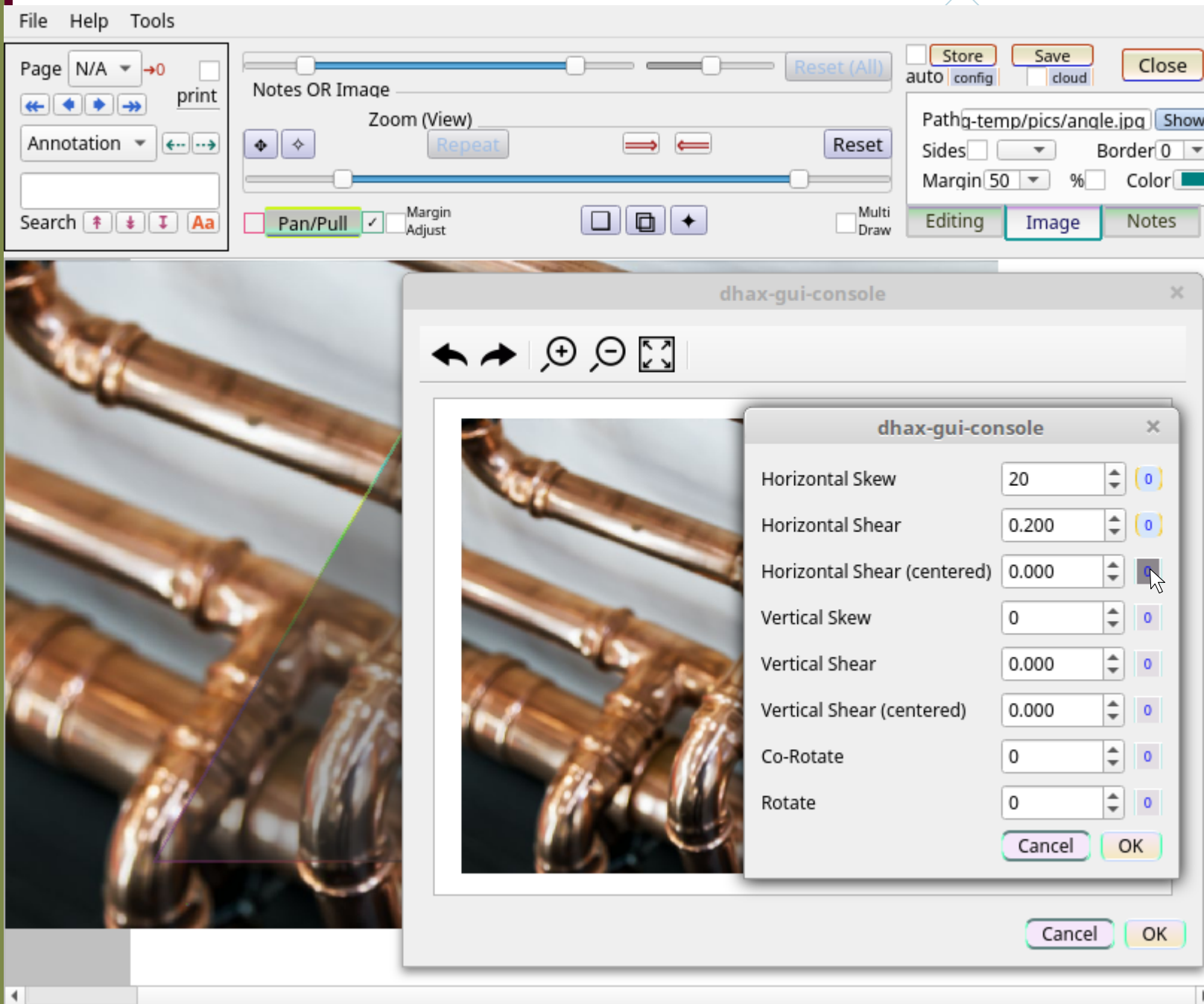
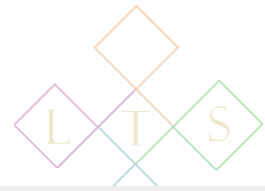


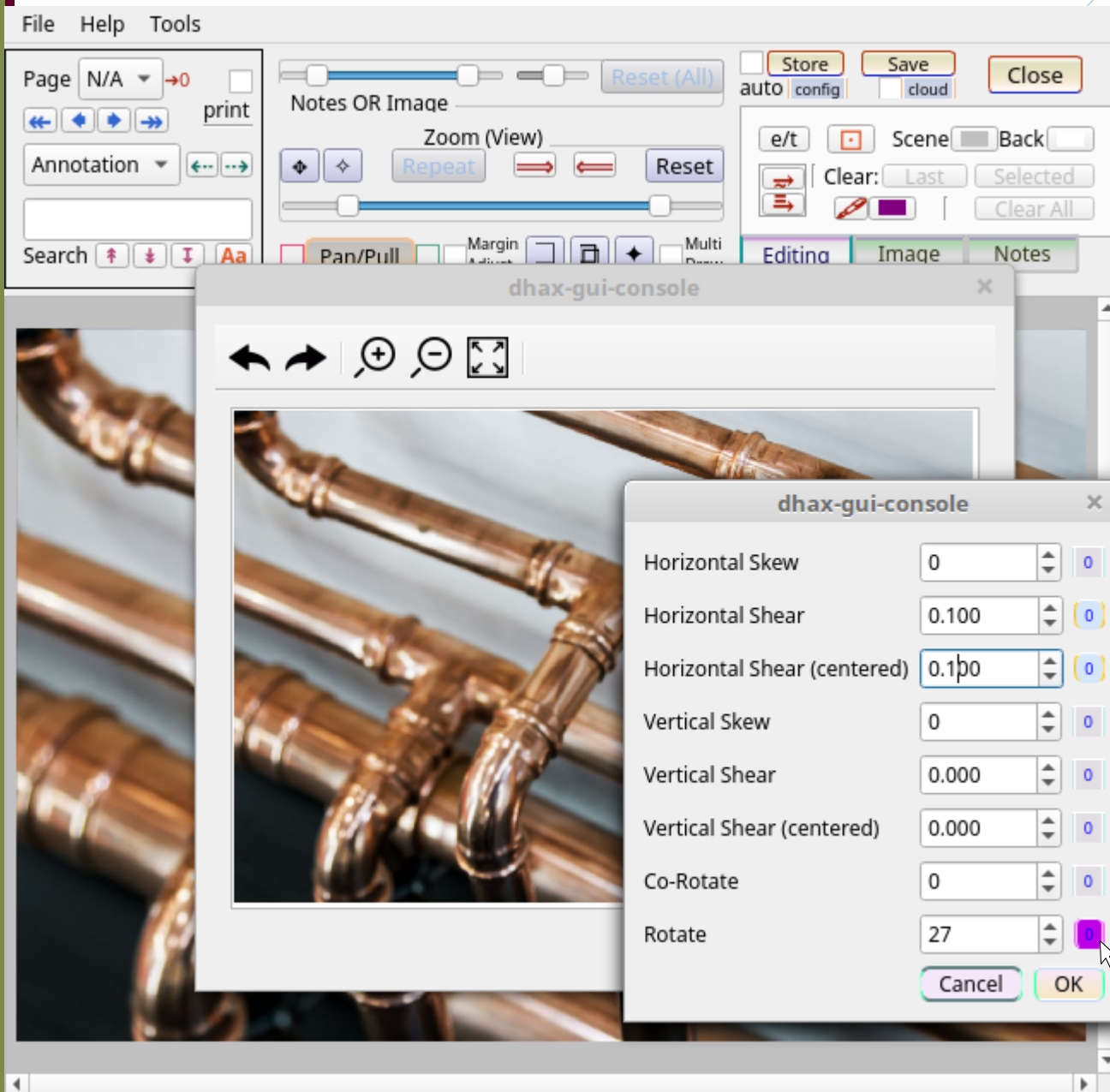
Image-editor tool to visualize different pipeline stages (slides 3-10 and 22-24)



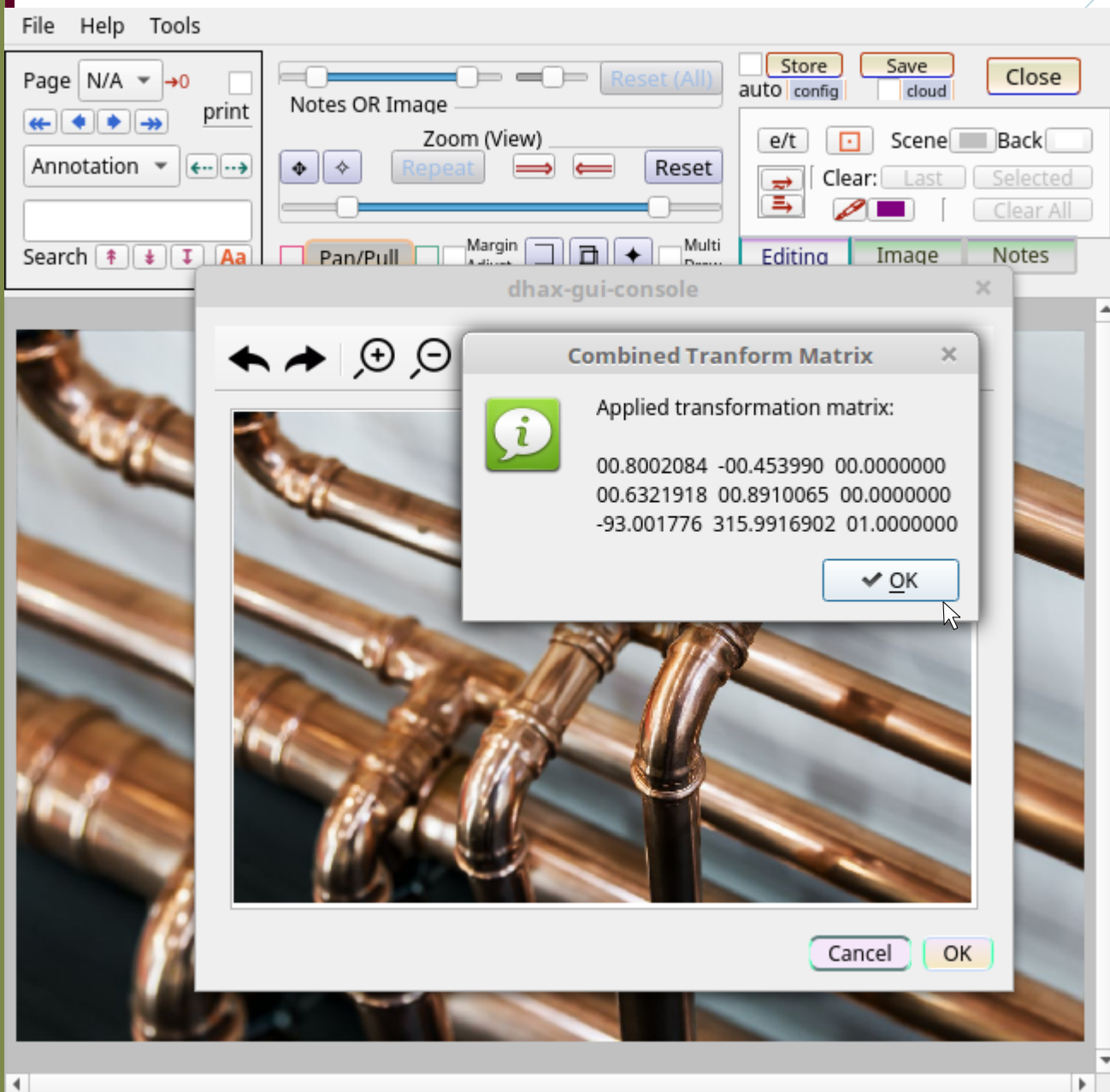
**Apply shear/rotate transform to linearize object proposals
(slides 4-8)**



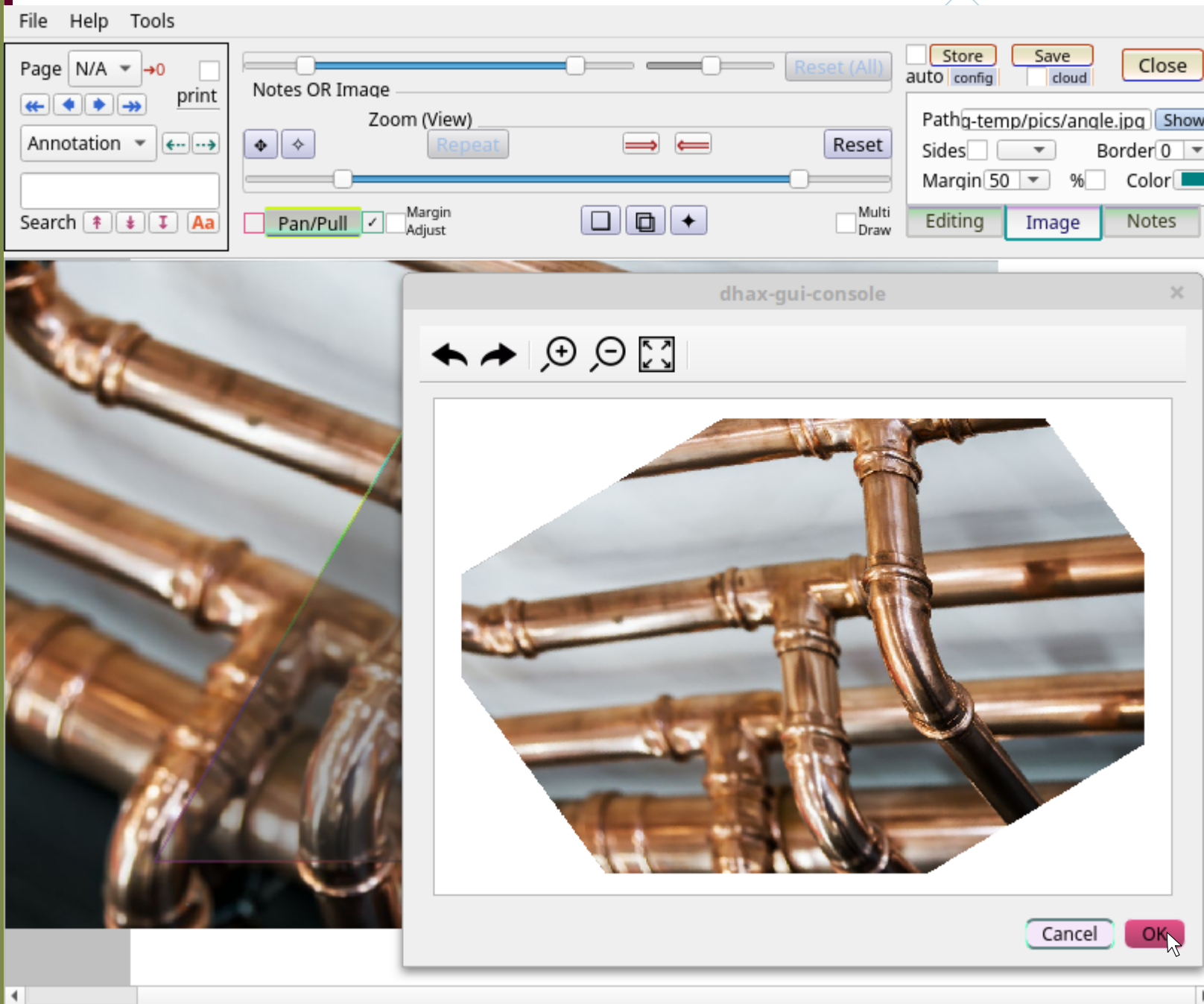
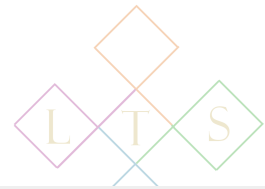
Custom dialog boxes for specific transforms and their parameters (slides 5, 6, 12-15, and 17)



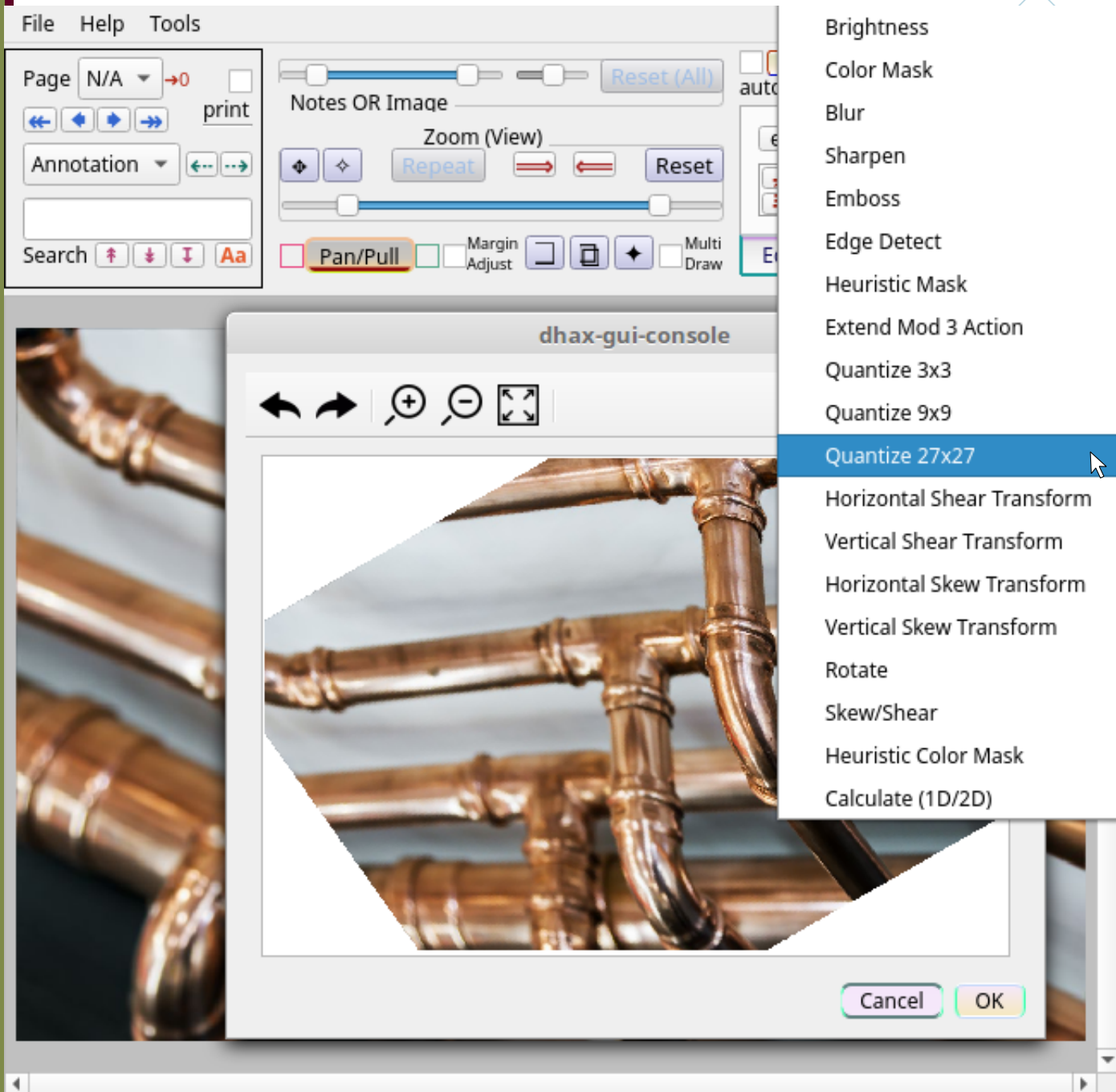
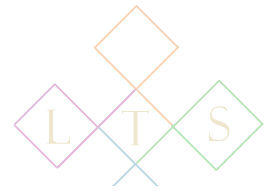
Shear, skew, and rotate transforms can be merged and/or defined through a common dialog box



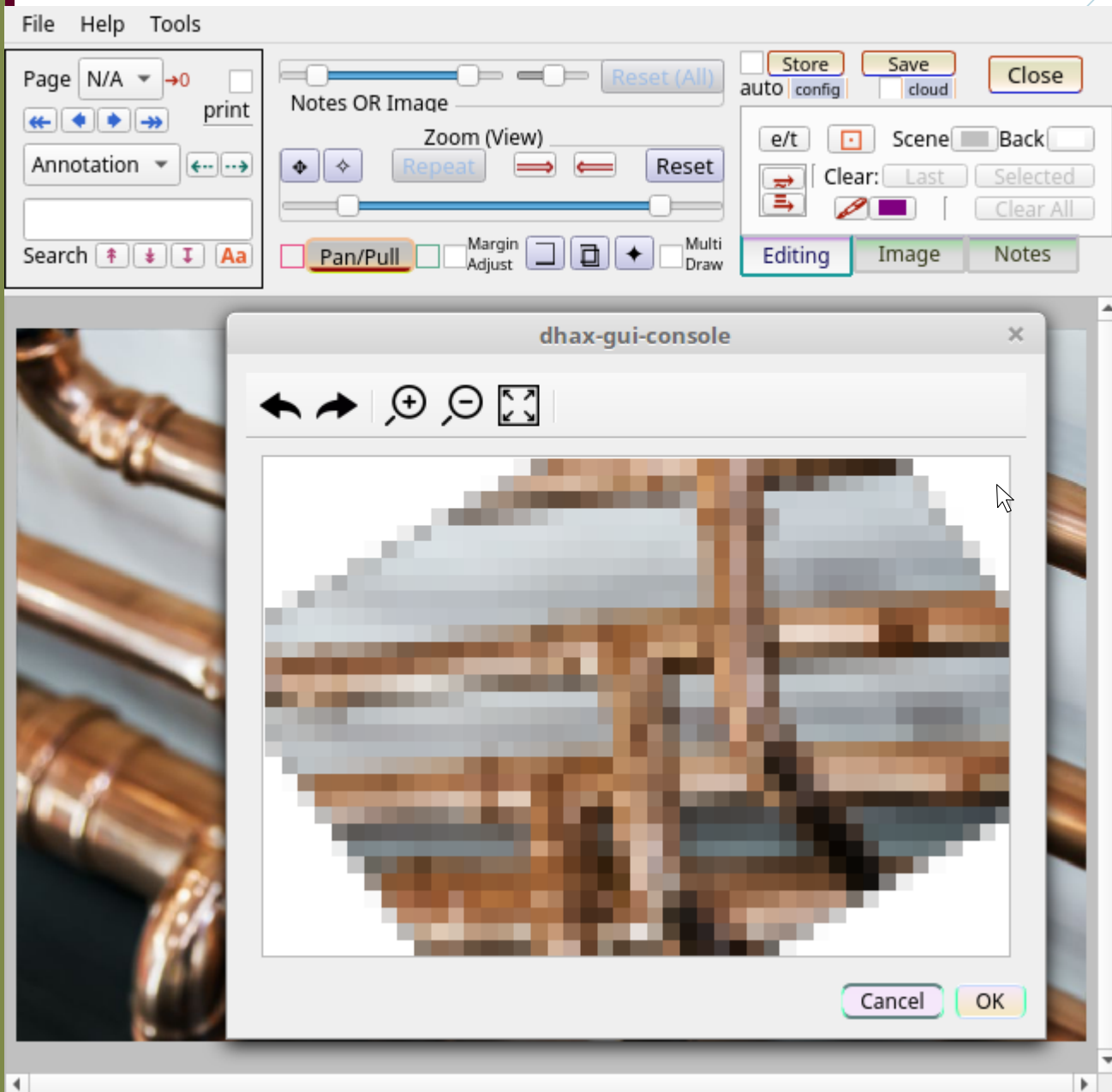
As a convenience during workflow development, users can see quantitative information about processing instructions created graphically; for example, the matrices generated by skew/shear/rotate transforms or combinations thereof



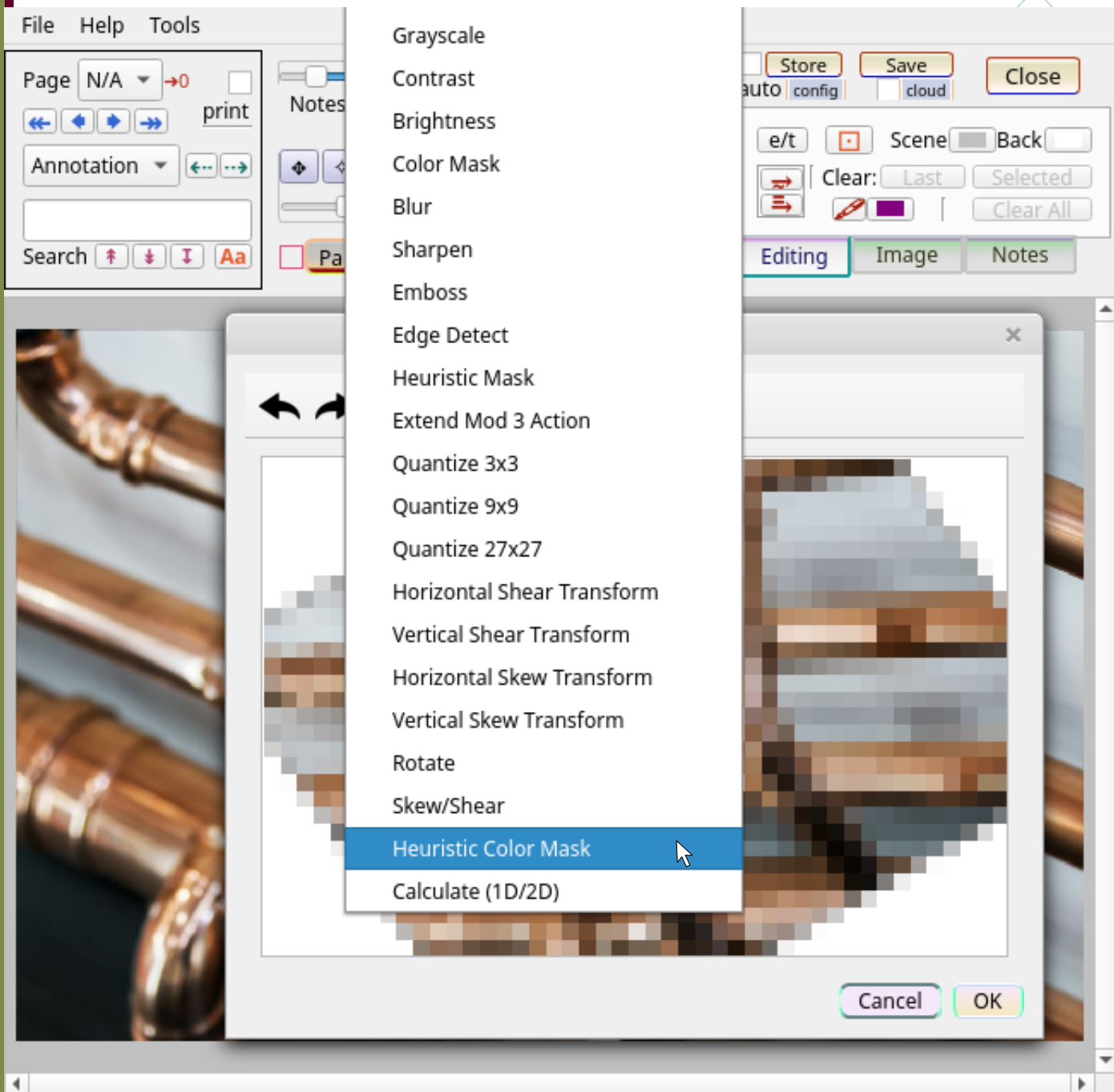
Visualizing the end-product of the transform whose matrix was reported on the previous slide (Slide 7)



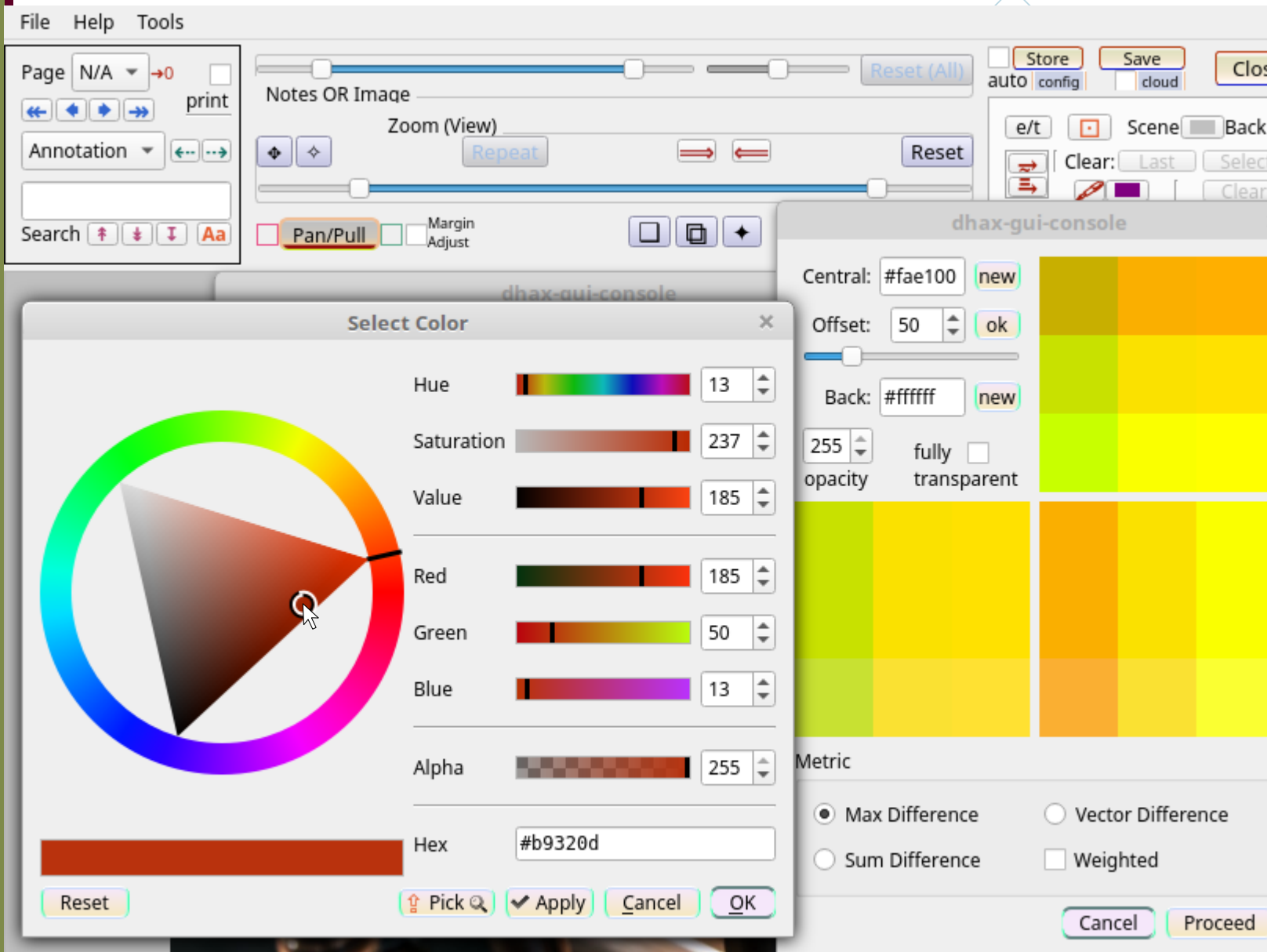
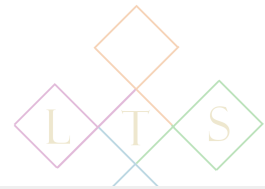
Discretize at 3-cube (slides 9-10)



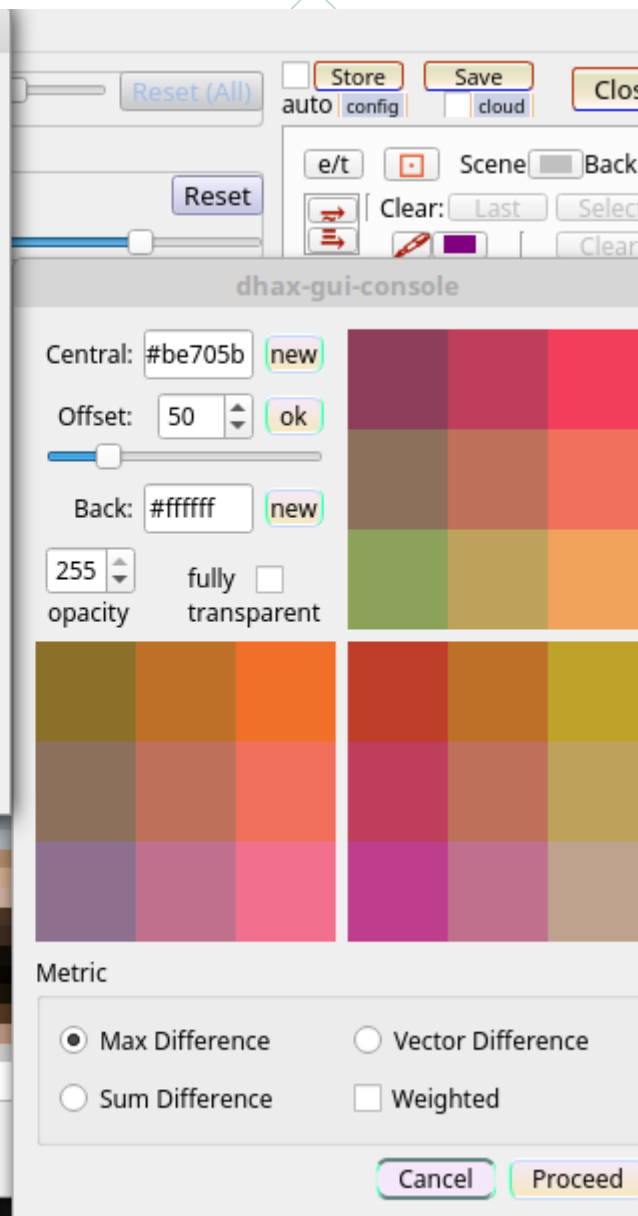
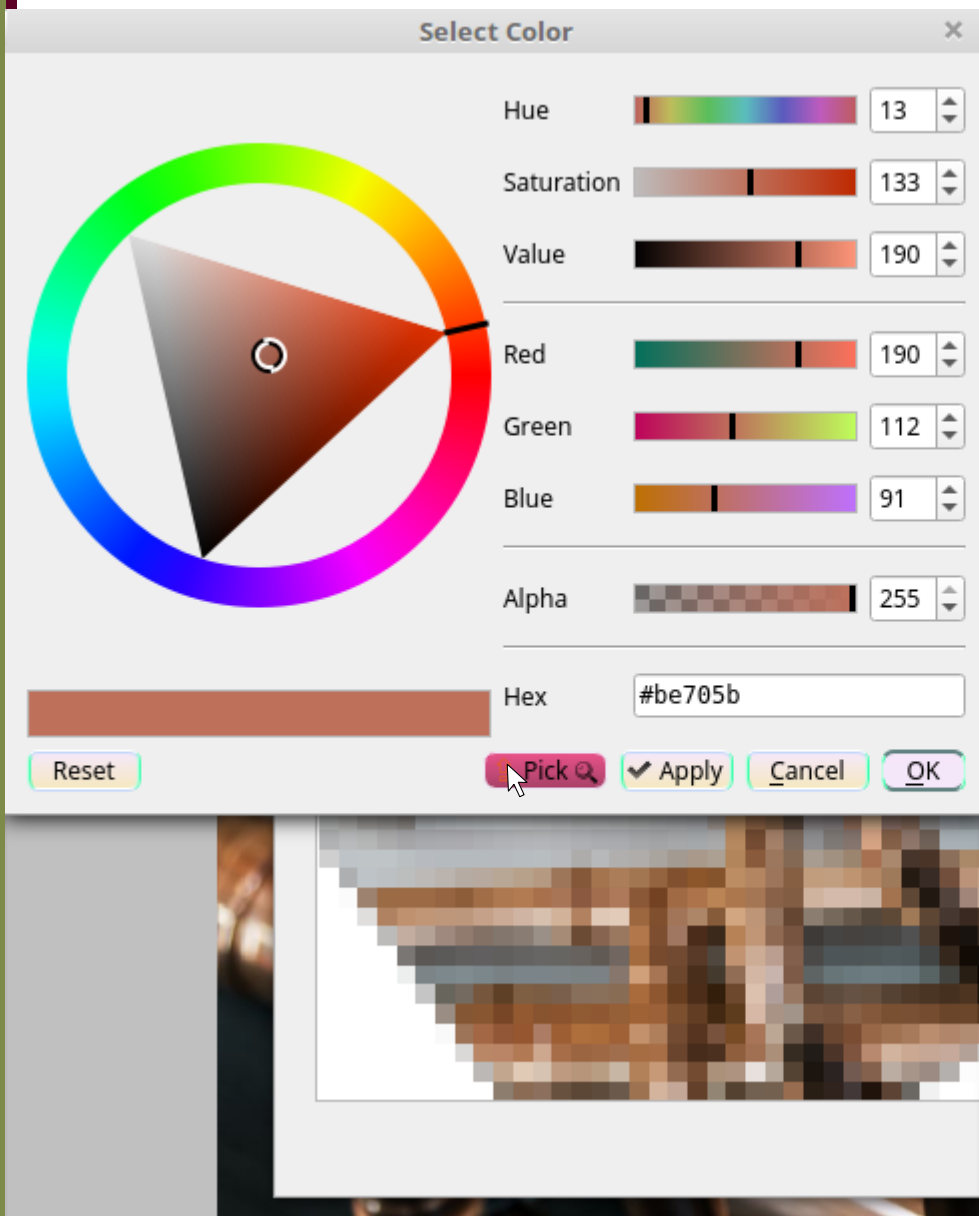
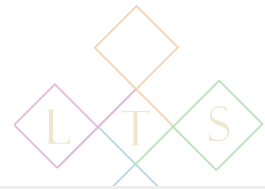
Result after a (27x27) discretization step, which averages color values within the bounds of an evenly-spaced grid



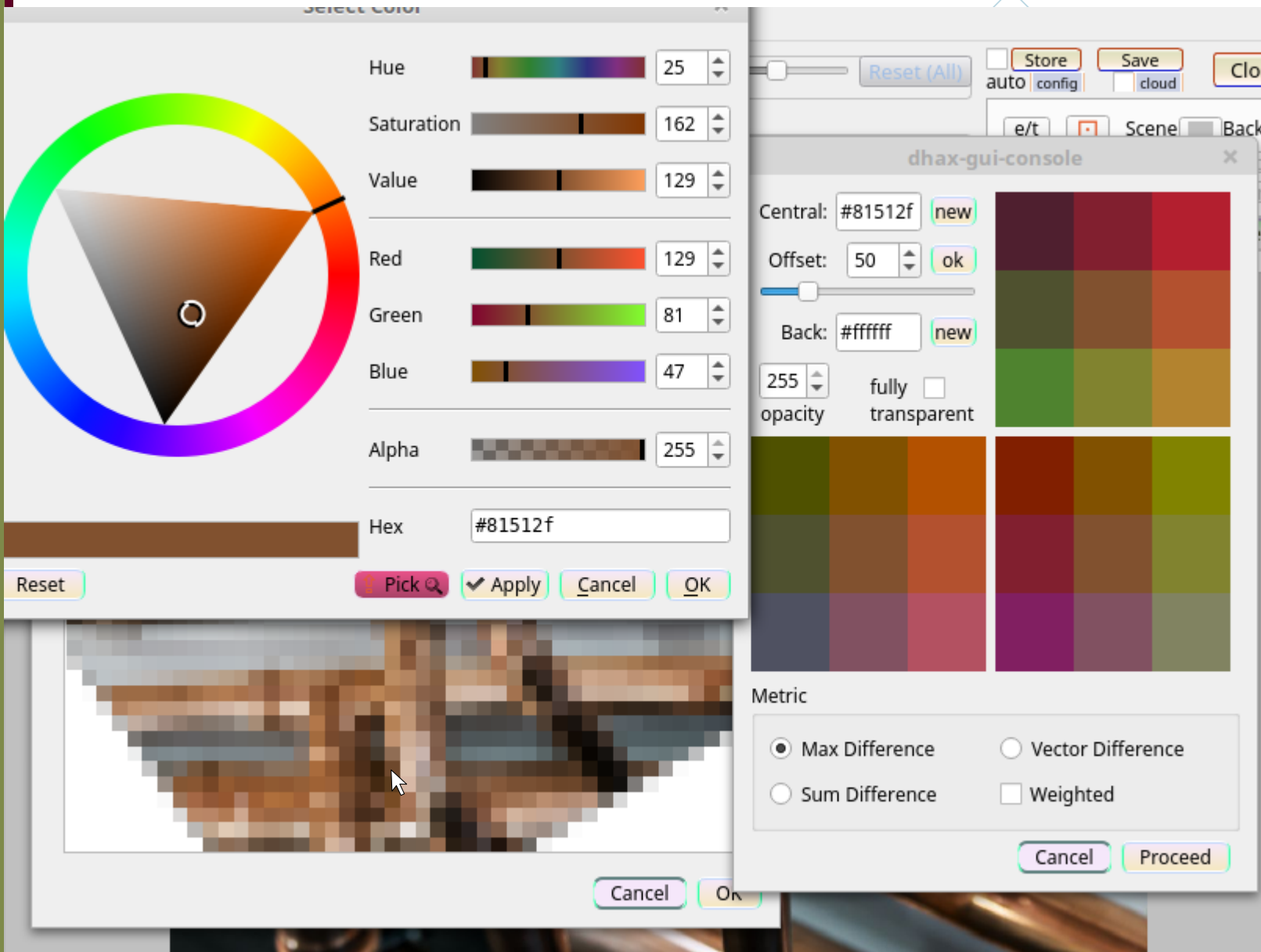
Apply color mask to isolate foreground from background (slides 11-16)



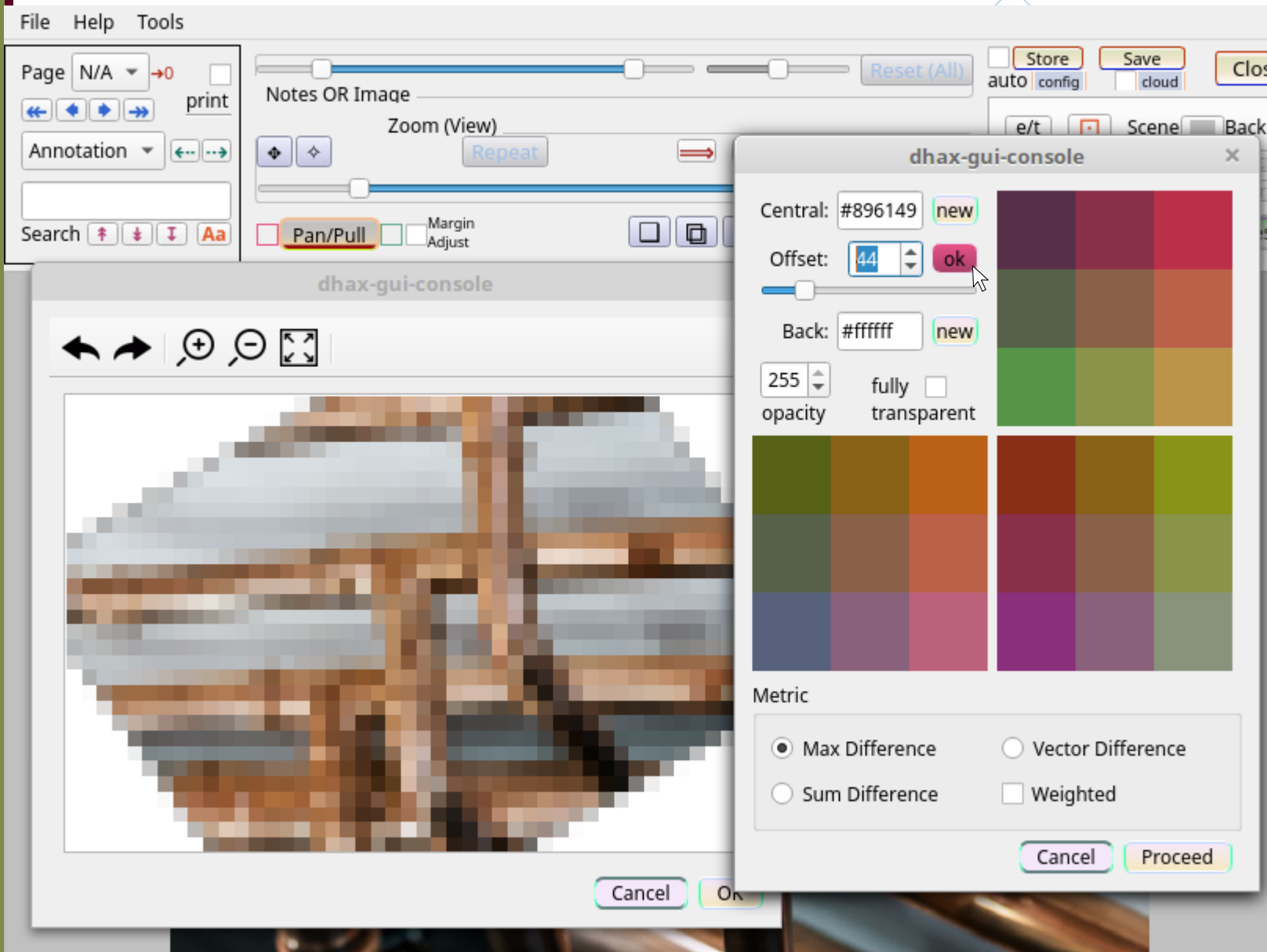
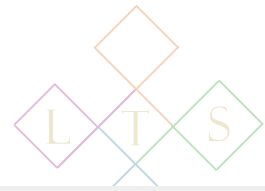
Select a color for the mask



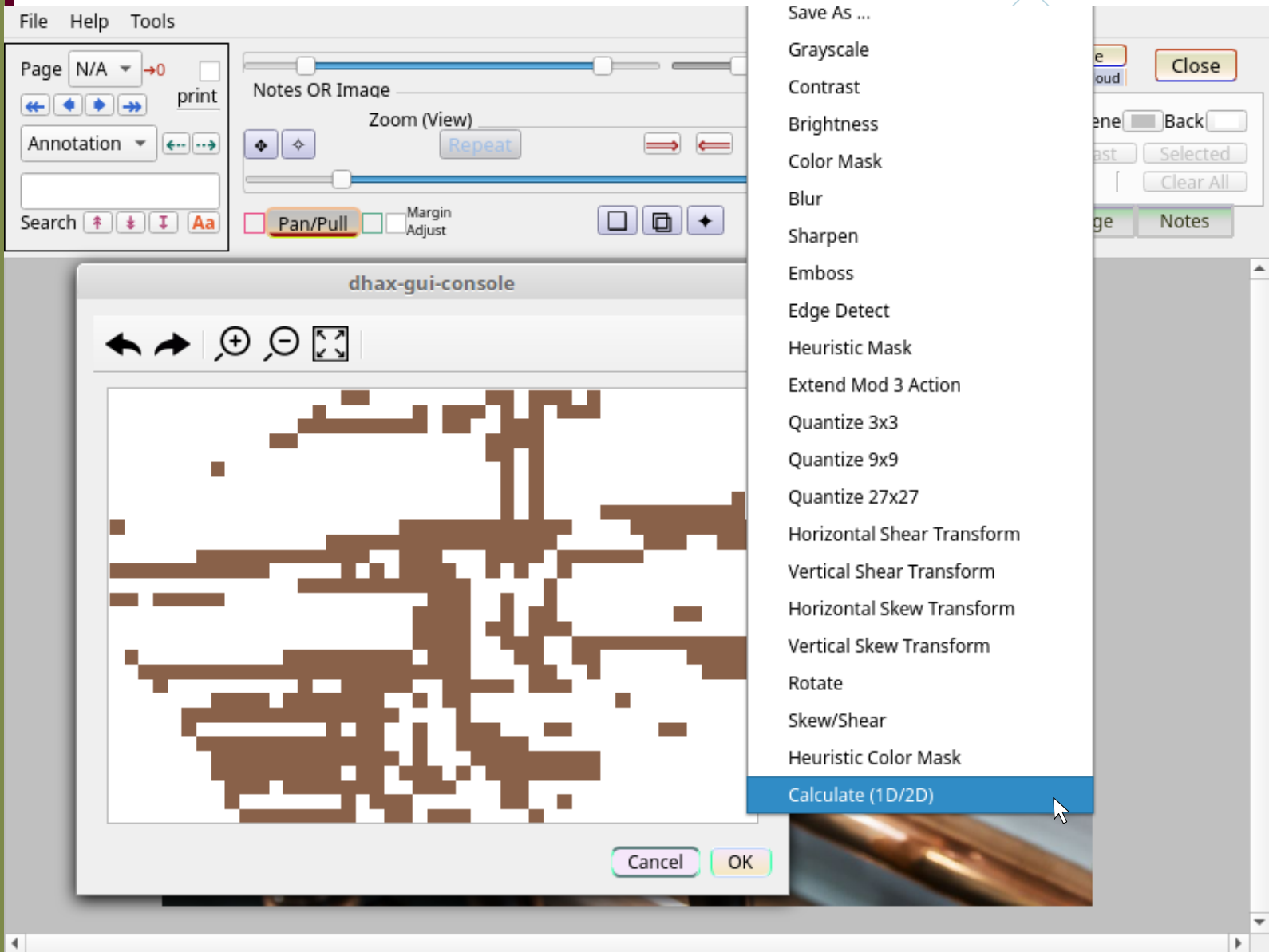
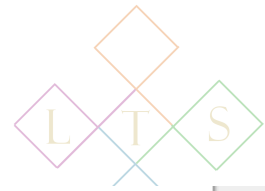
Color-picker from anywhere on screen to grab foreground color for heuristic masks



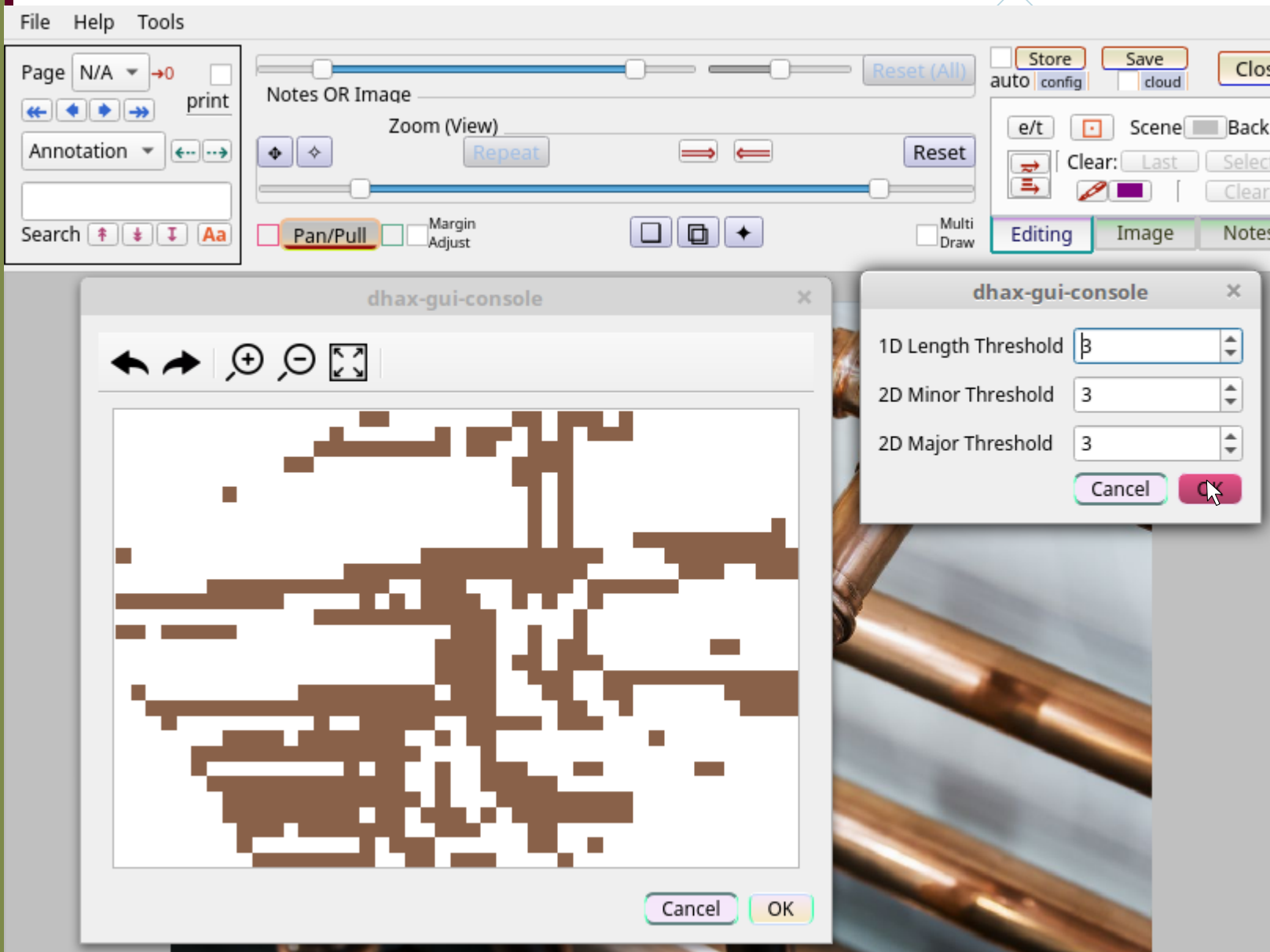
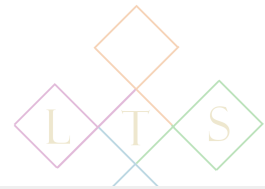
One sensible option is to grab a visually prominent foreground color directly from the image being analyzed



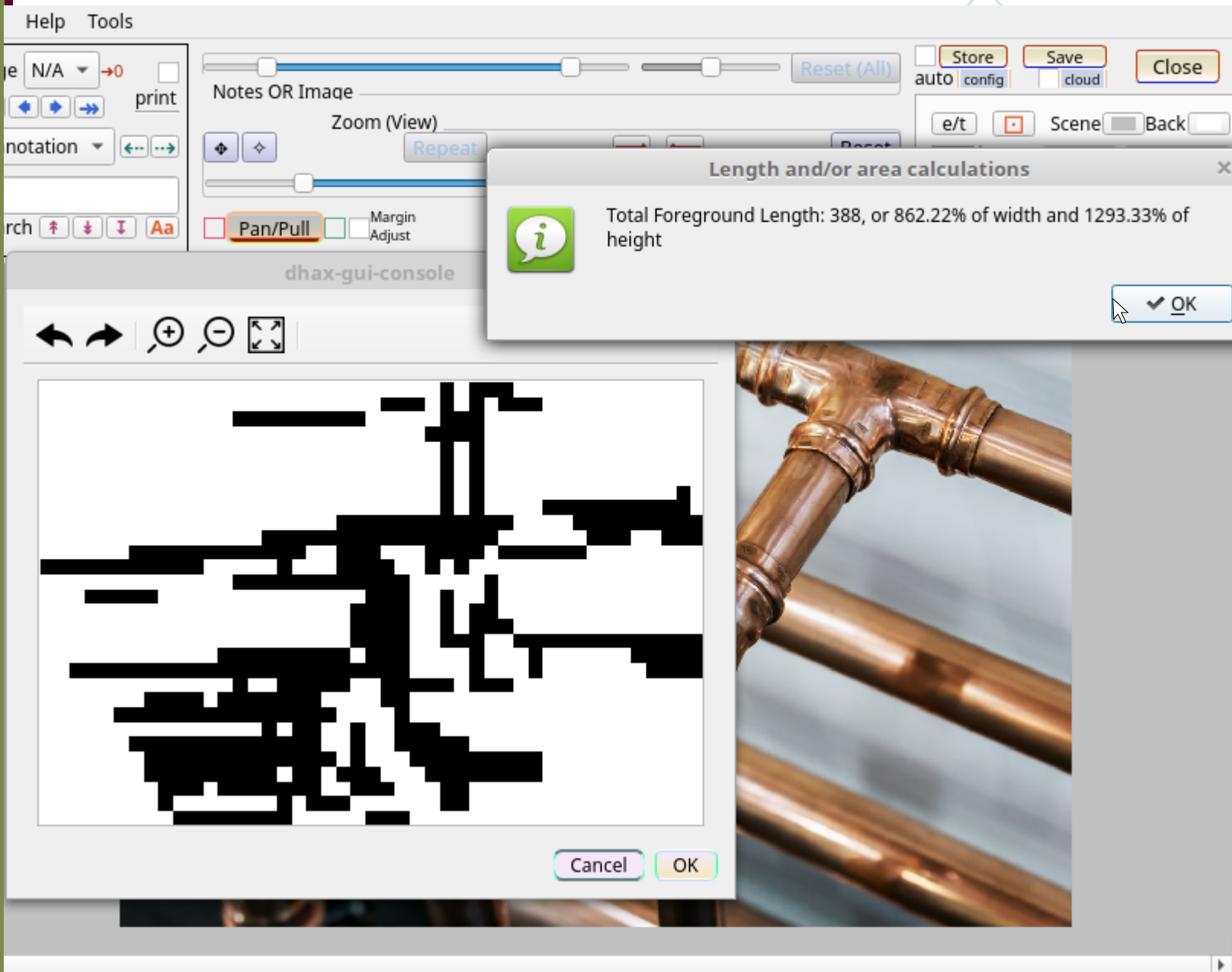
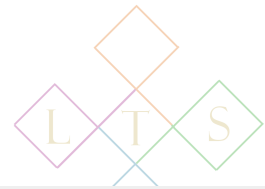
Adjust the offset value, which marks how far a color may deviate from the central prototype while still being classified as foreground



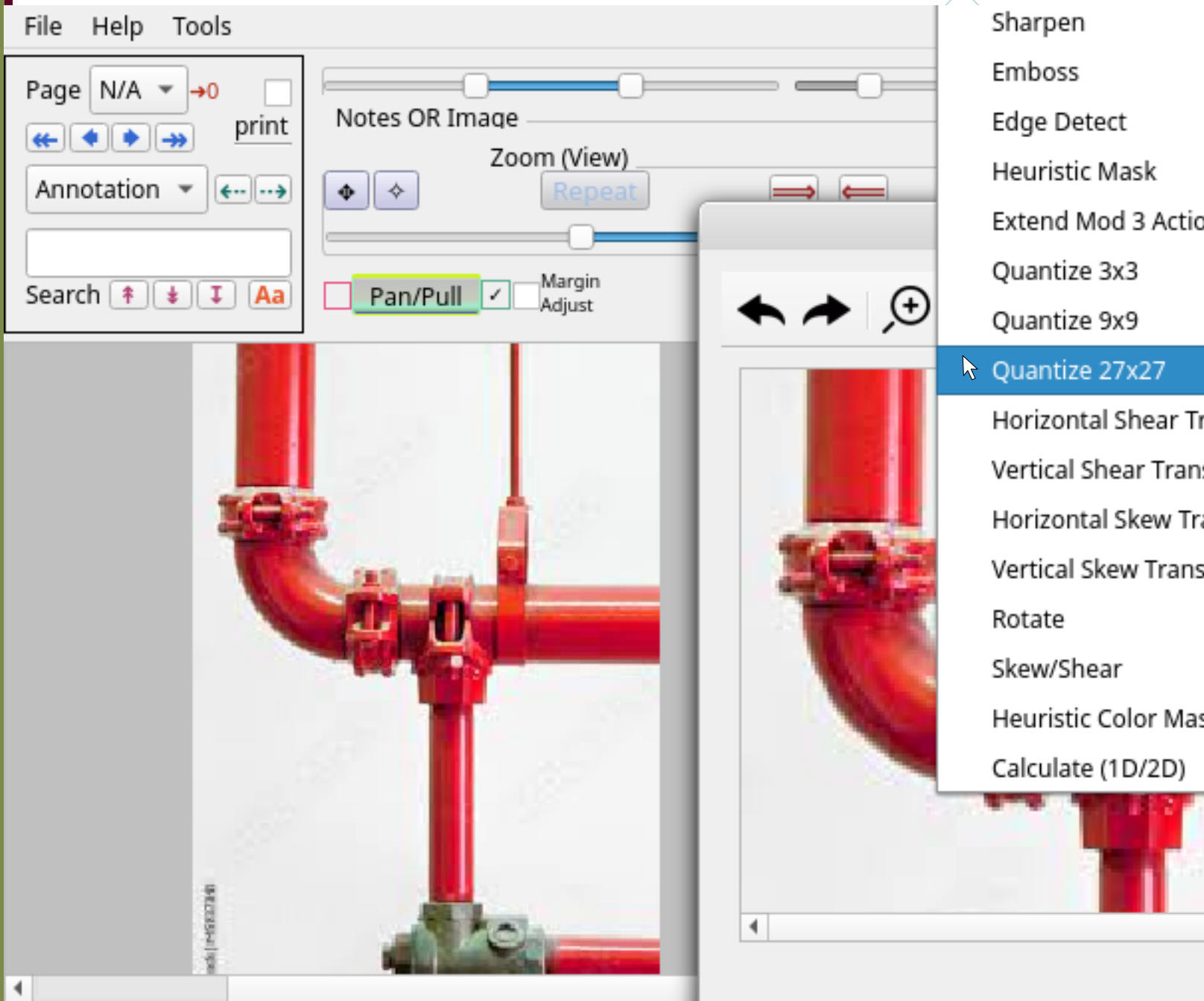
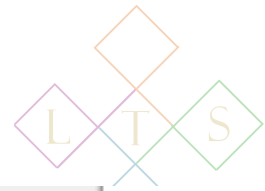
Erode foreground to line segments at 3-cube scale (slides 16-18)



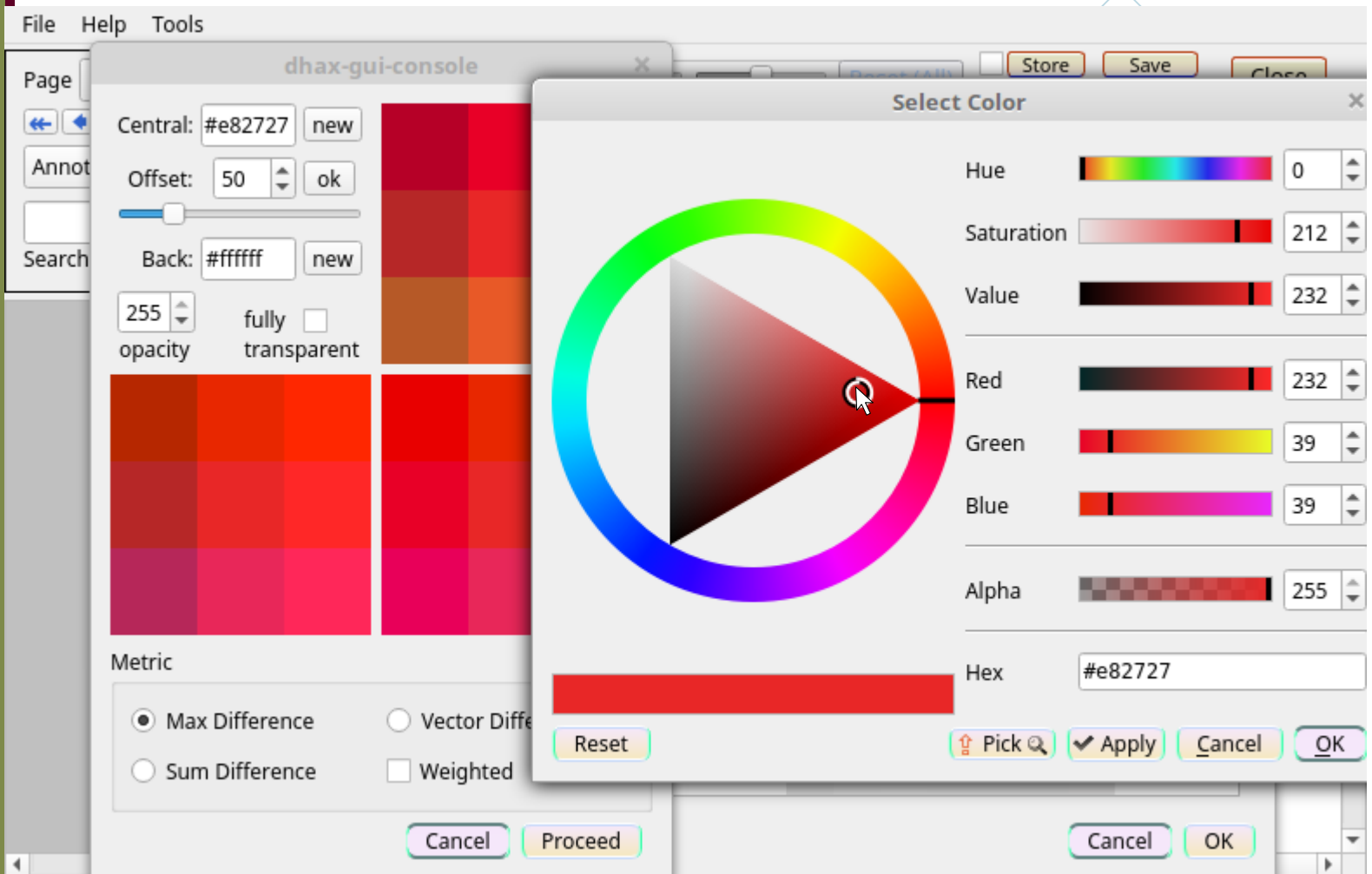
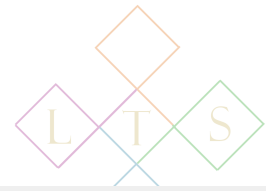
Filter line segments and/or rectangular regions by length or width/height



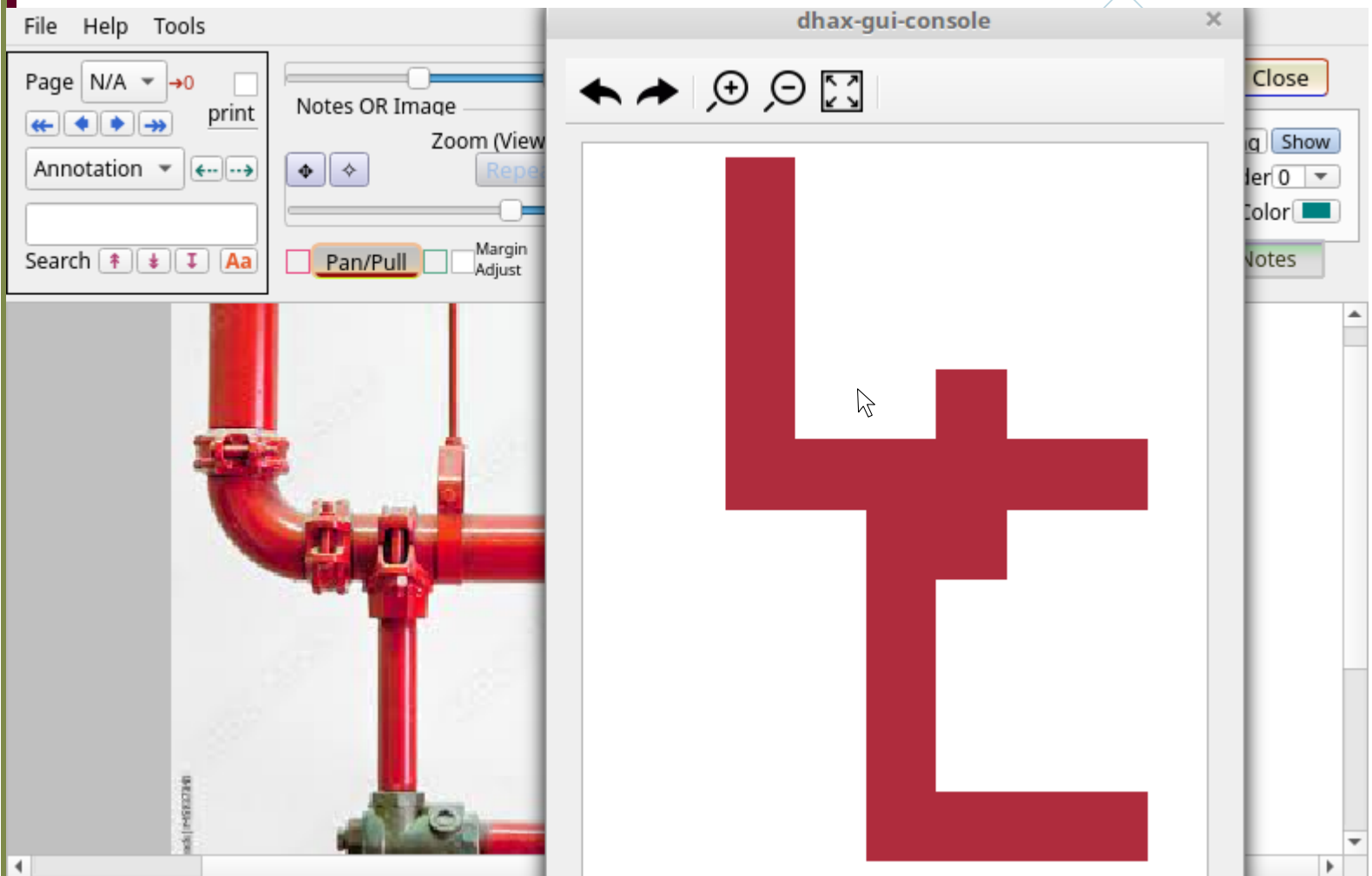
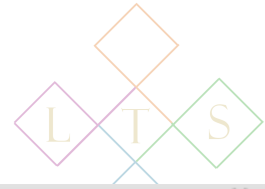
Calculate total length of remaining segments, as percentage of image dimensions



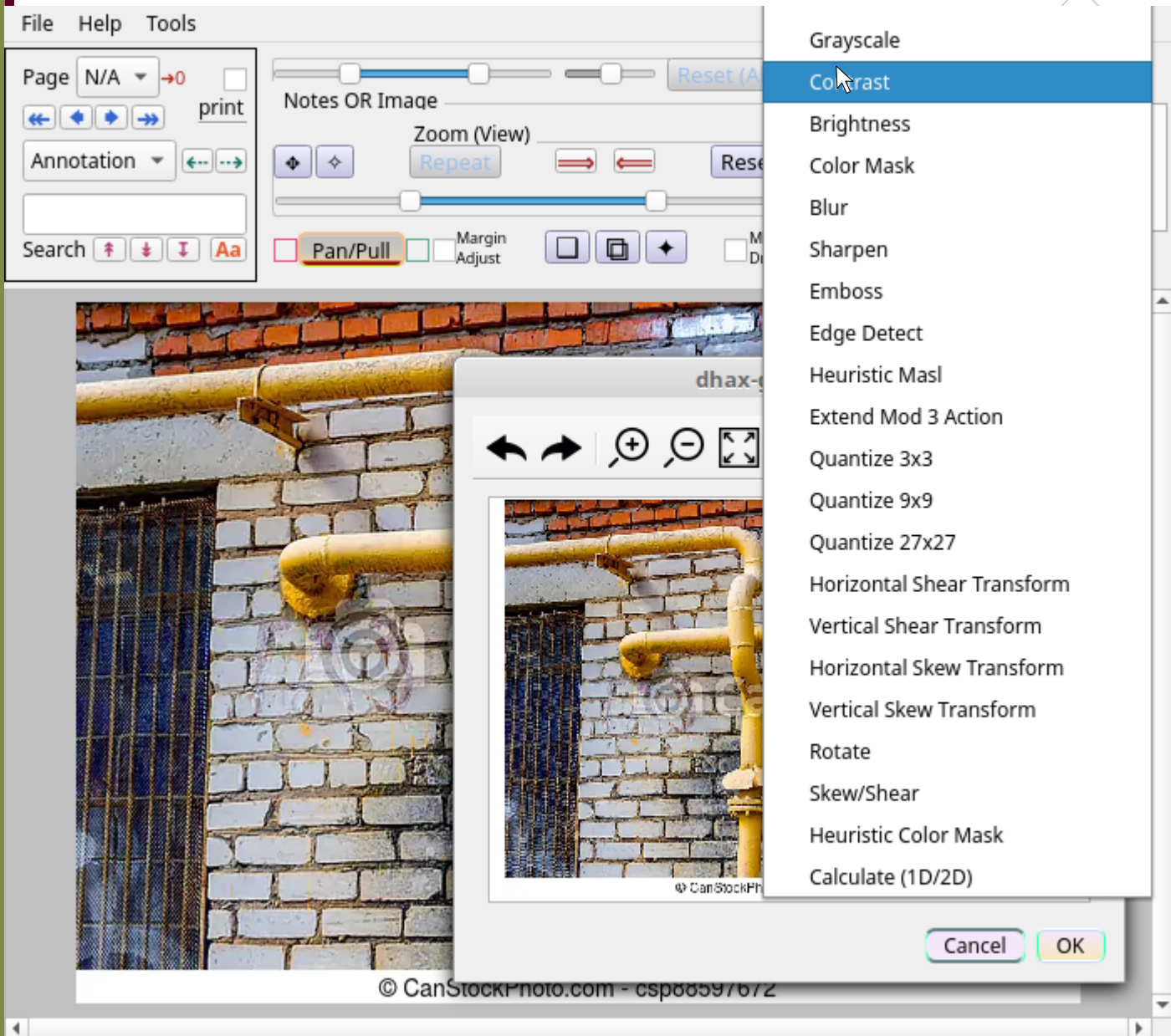
Additional Example Pipeline: without coordinate rotations (slides 19-21)



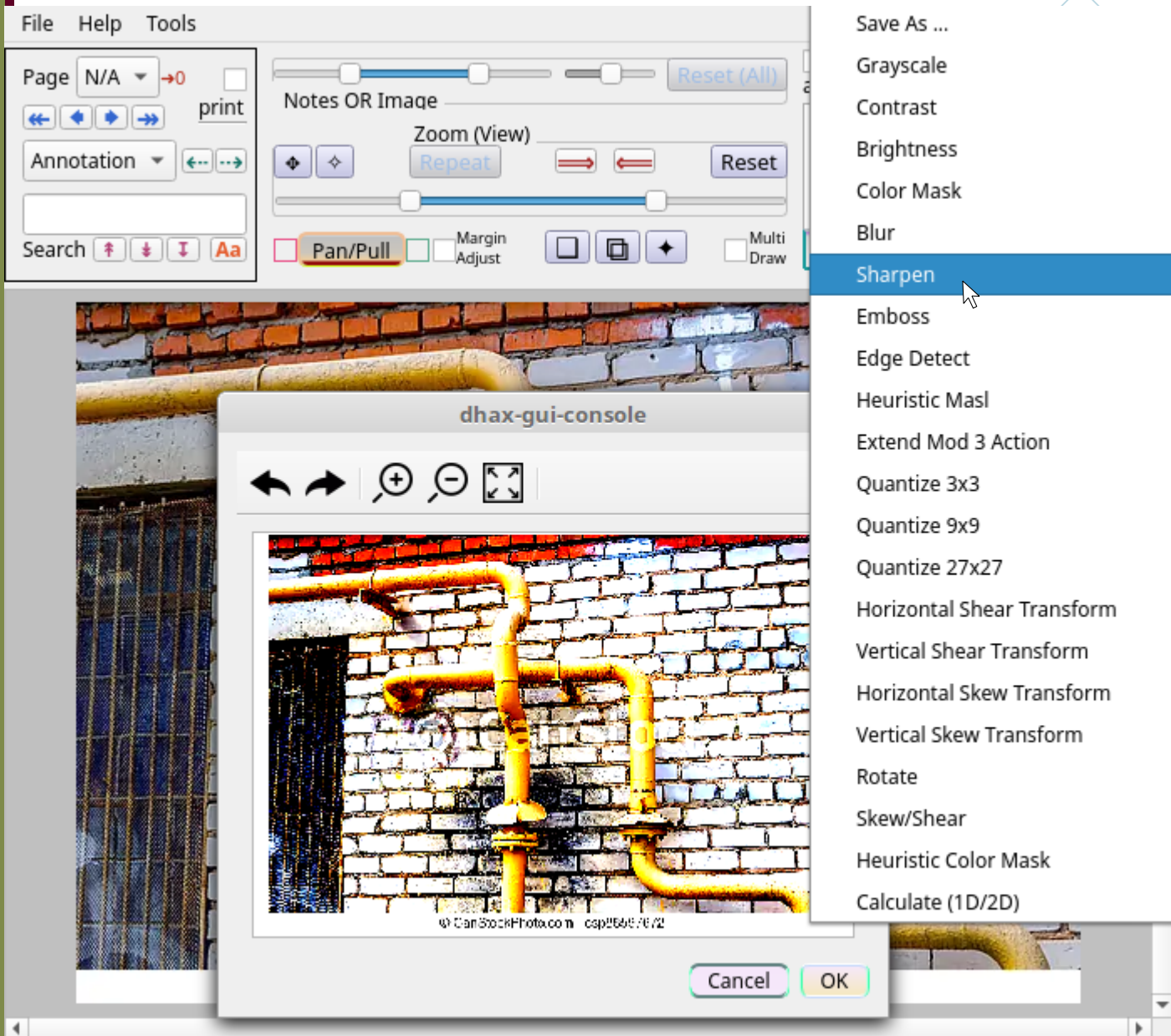
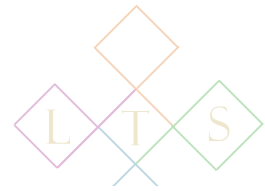
This example calls for a wider foreground-tolerance offset due to the regions of interest showcasing purer colors



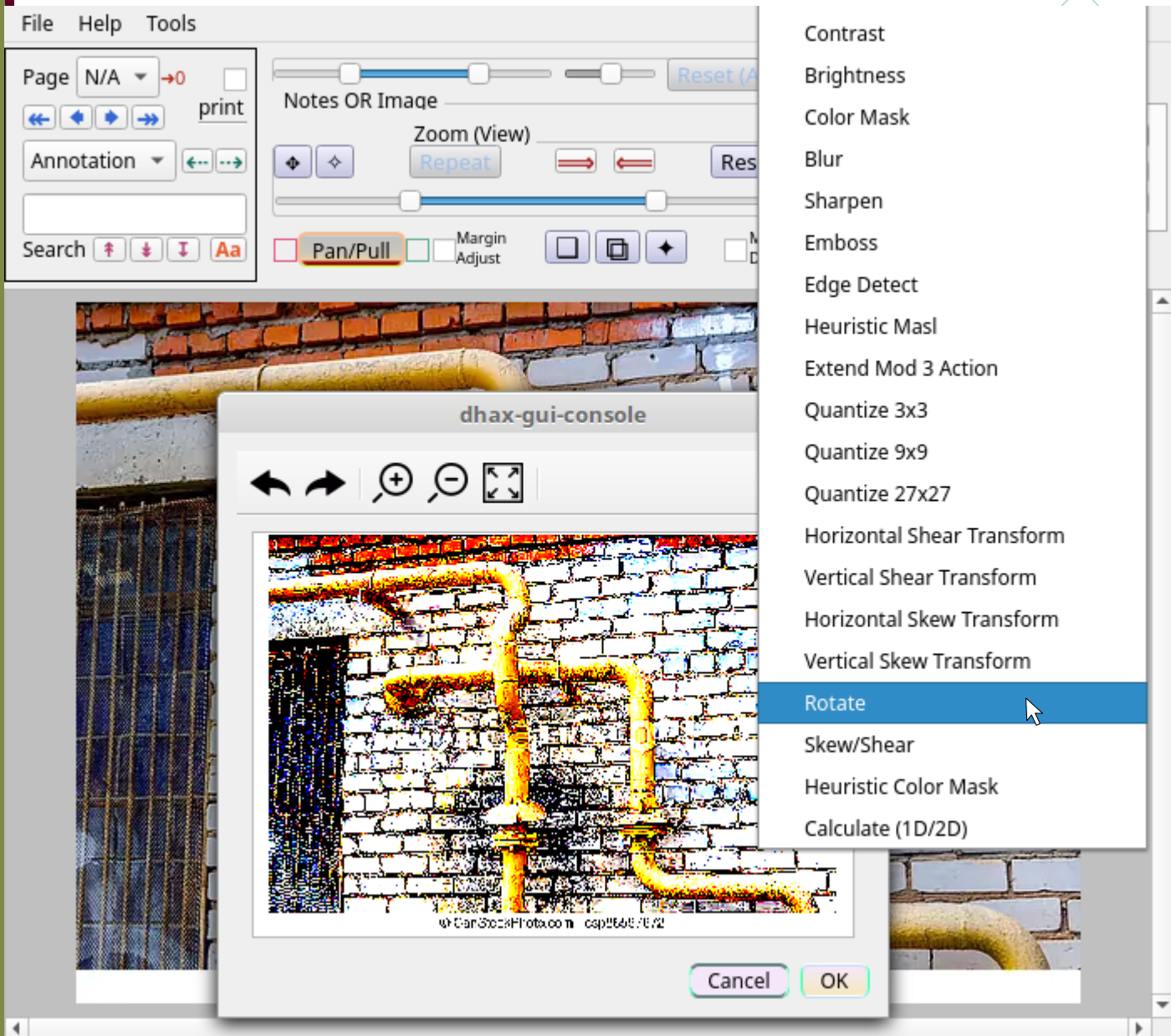
Discretized foreground retains the rough shape of the object-region — segment lengths can be calculated using the same commands as on Slide 18 (above)



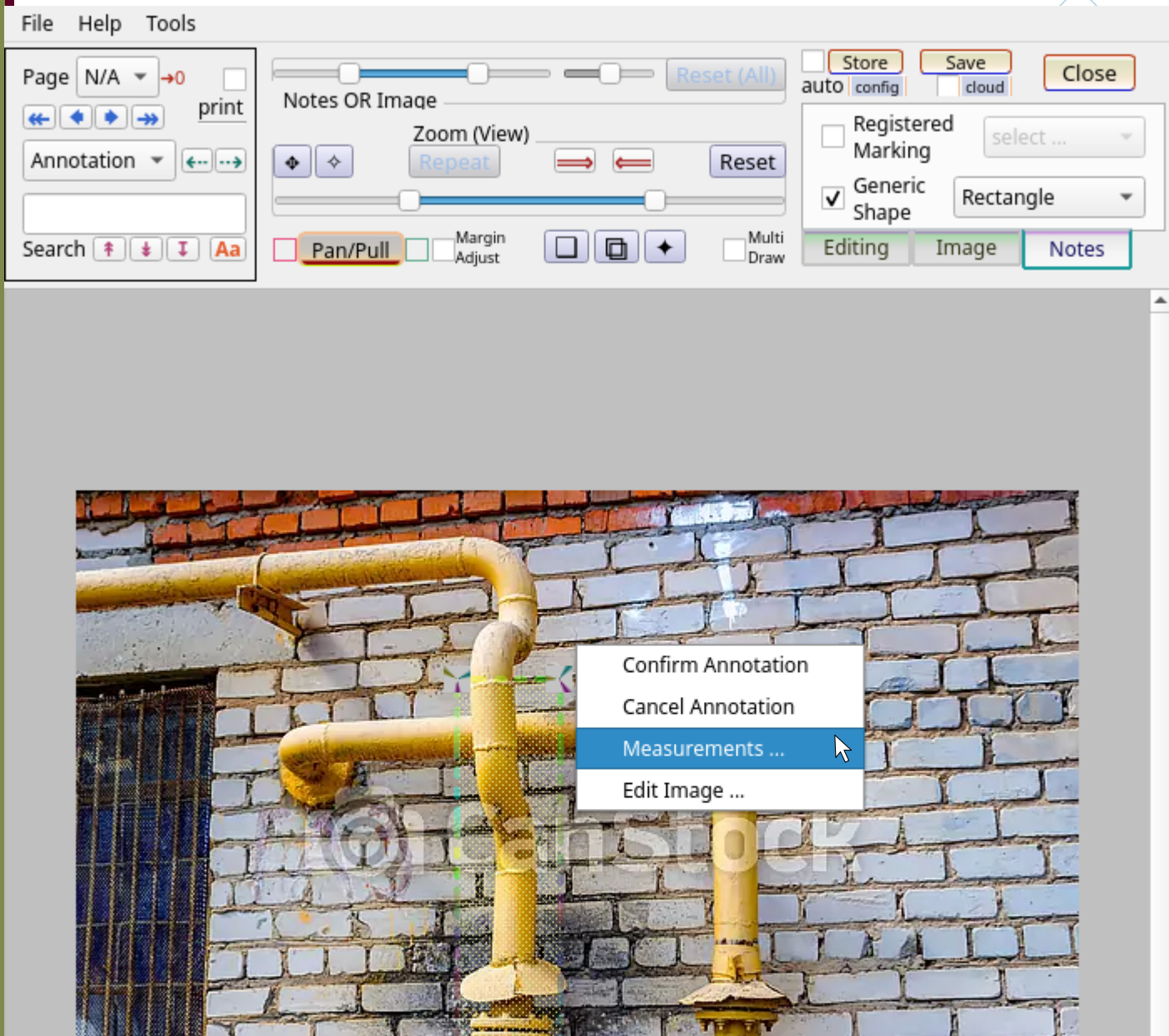
Applying contrast-highlights to accentuate foreground differentiation, as one example of early-pipeline transforms that may improve accuracy



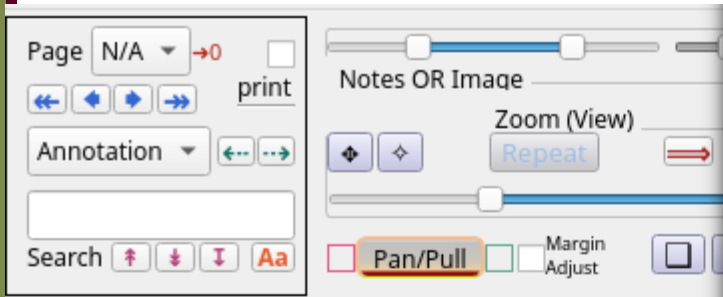
“Sharpen” effects are another early-pipeline foreground reinforcement possibility



Contrast/sharpen transforms can be chained with rotations as precursors to discretizing or segmentation steps



Region-of-Interest Parameters: geometric properties, measurements in pixels and image-dimension proportions (slides 25-26); color properties (slides 27-28); visualize annotation shape as color mask and visualize color-metric table (slide 28)



© CanStockPhoto.com

Width	61
Height	272
Perimeter	666
Image Width	600
Image Height	419
Image Perimeter	2038
Area	16592
Image Area	251400
Width/Height	0.224265
Height/Width	4.45902
Image Width/Height	1.43198
Image Height/Width	0.698333
Width (H)	0.101667
Height (H)	0.649165
Perimeter (H)	0.326791
Area (H)	0.0659984
Left Margin	228
Top Left Margin	32376
Top Margin	113
Top Right Margin	54745
Right Margin	311
Bottom Right Margin	48938
Bottom Margin	34
Bottom Left Margin	26570
Left Margin (H)	0.379368
Top Left Margin (H)	5.74348
Top Margin (H)	0.269048
Top Right Margin (H)	9.71173
Right Margin (H)	0.517471
Bottom Right Margin (H)	8.68157
Bottom Margin (H)	0.0809524
Bottom Left Margin (H)	4.7135
Occurant Color Mean (10 iterations)	Hex: c56e12
Additive Color Mean	Hex: a08658

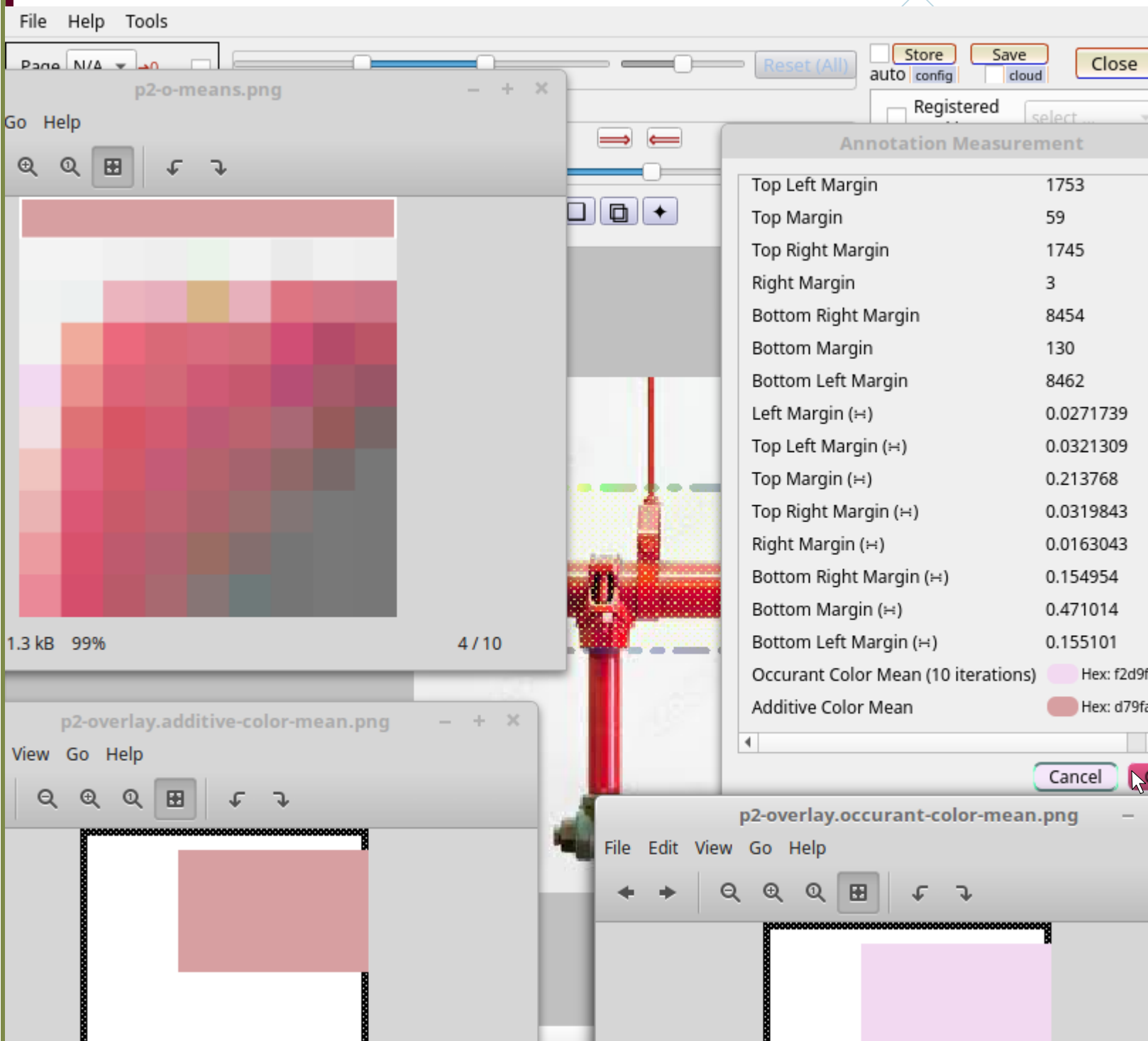
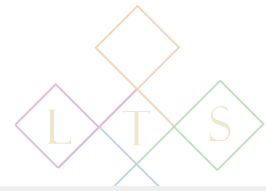
Measurements provide both geometric and chromatic feature vectors

The screenshot displays a software interface with a red pipe image. An 'Annotation Measurement' dialog box is open, showing the following data:

Top Left Margin	1690
Top Margin	58
Top Right Margin	1700
Right Margin	6
Bottom Right Margin	9540
Bottom Margin	138
Bottom Left Margin	9530
Left Margin (H)	0.0217391
Top Left Margin (H)	0.0309762
Top Margin (H)	0.210145
Top Right Margin (H)	0.0311595
Right Margin (H)	0.0326087
Bottom Right Margin (H)	0.17486
Bottom Margin (H)	0.5
Bottom Left Margin (H)	0.174676
Occurant Color Mean (10 iterations)	Hex: e5d9e4
Additive Color Mean	Hex: d69a9d

At the bottom of the interface, a blue bar contains two buttons: 'Generate Overlay File' and 'Generate Occurant Color Mean Summary File'.

Use annotation measurements to visualize color-metric options (slides 27-28)

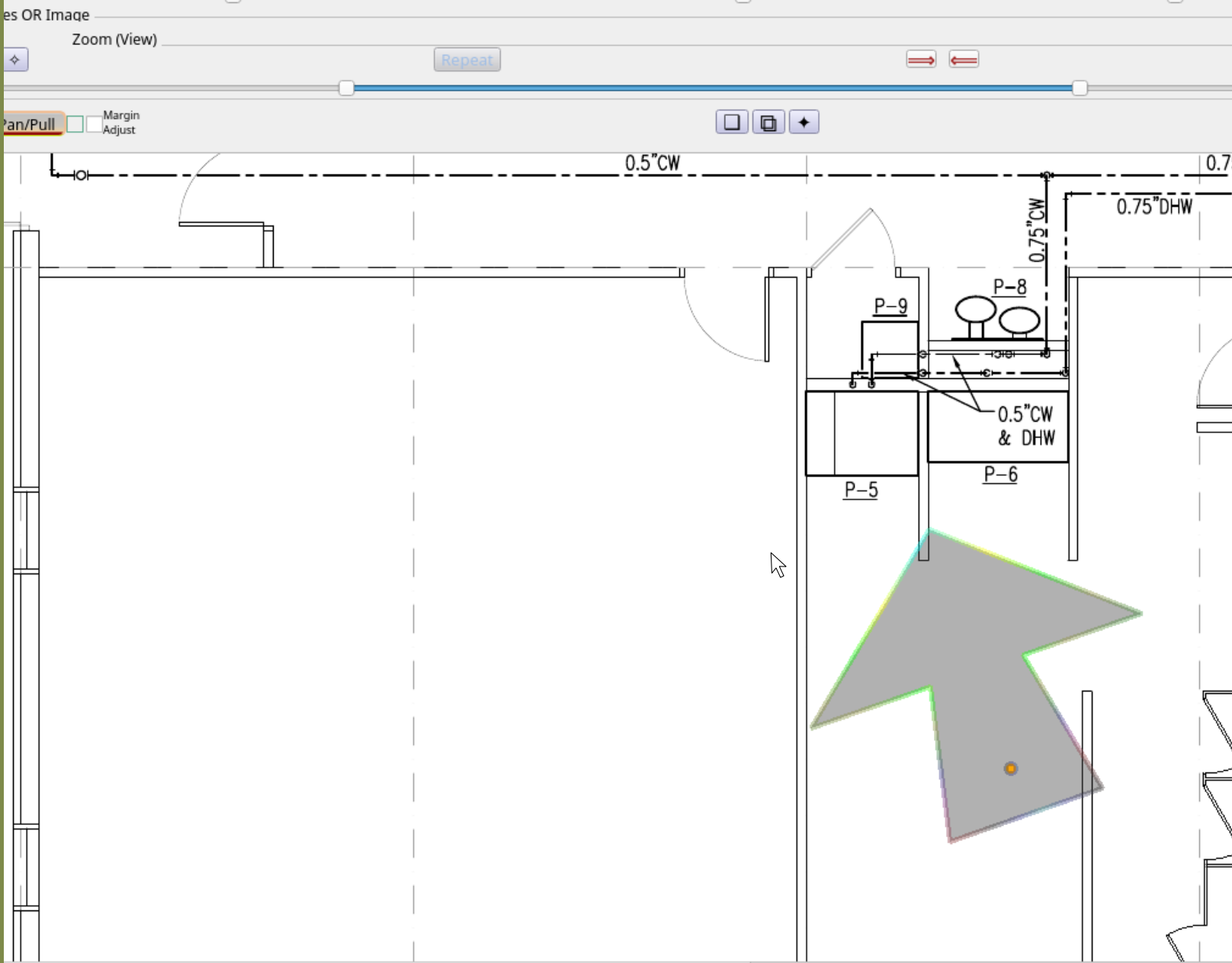
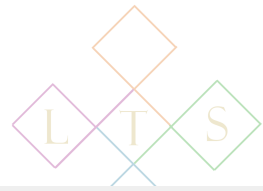


Visualizing color-means calculated according to different parameters, which also presents a visual outline of an annotated shape relative to its overall target image

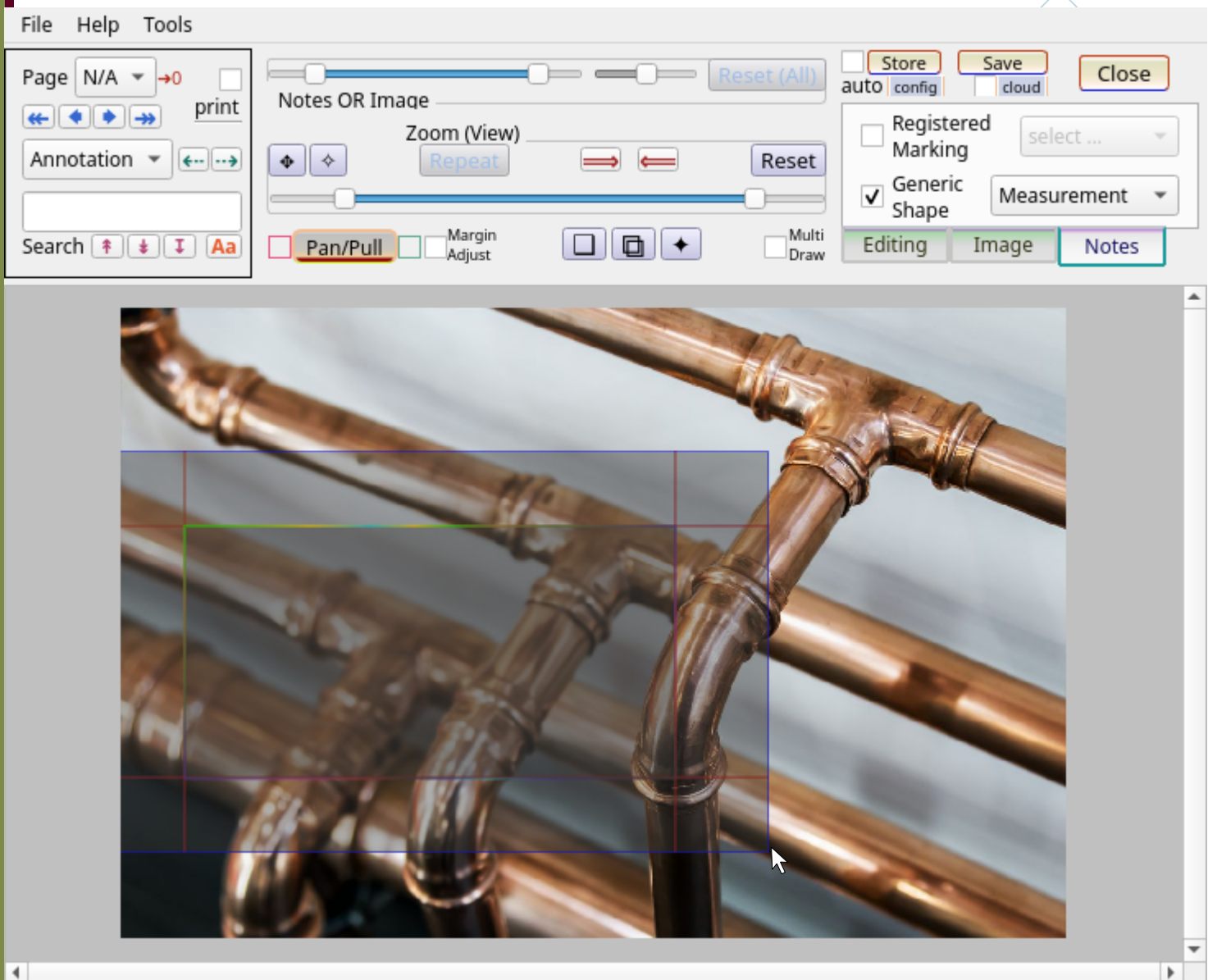


Special Annotation Tools: define special purpose shapes with multiple mouse phases, such as “multilines” (slide 29); rhomboids — for instance, to define skew/rotate parameters (slides 1-2), arrows with rotation and different shaft shapes (slide 30); measurement reporting templates (slides 31-32)

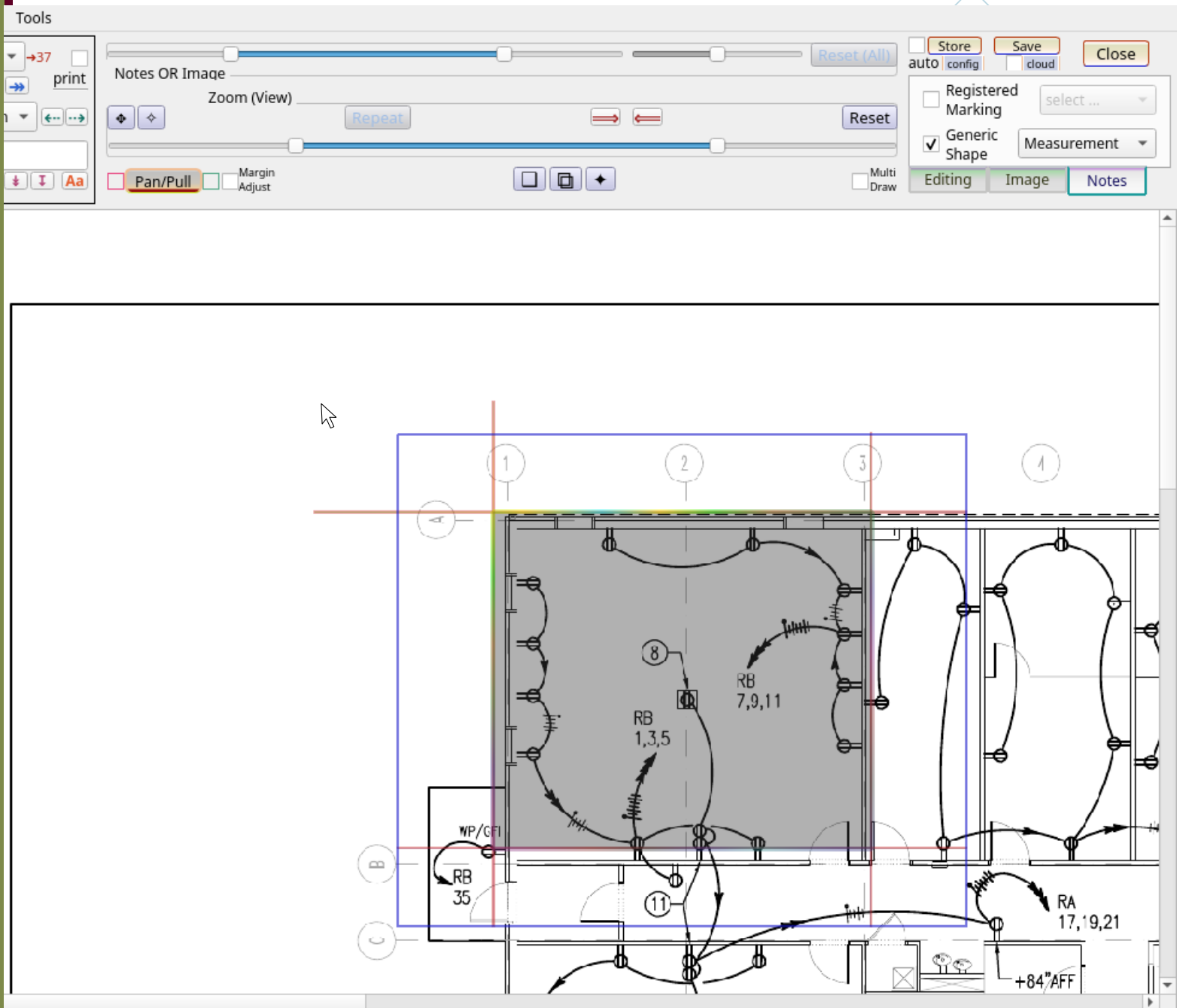
Extensible system for defining User Interface pragmas (specifically, custom sequences of user actions via mouse position/buttons and/or keyboard modifiers) to create annotations with mouse gestures



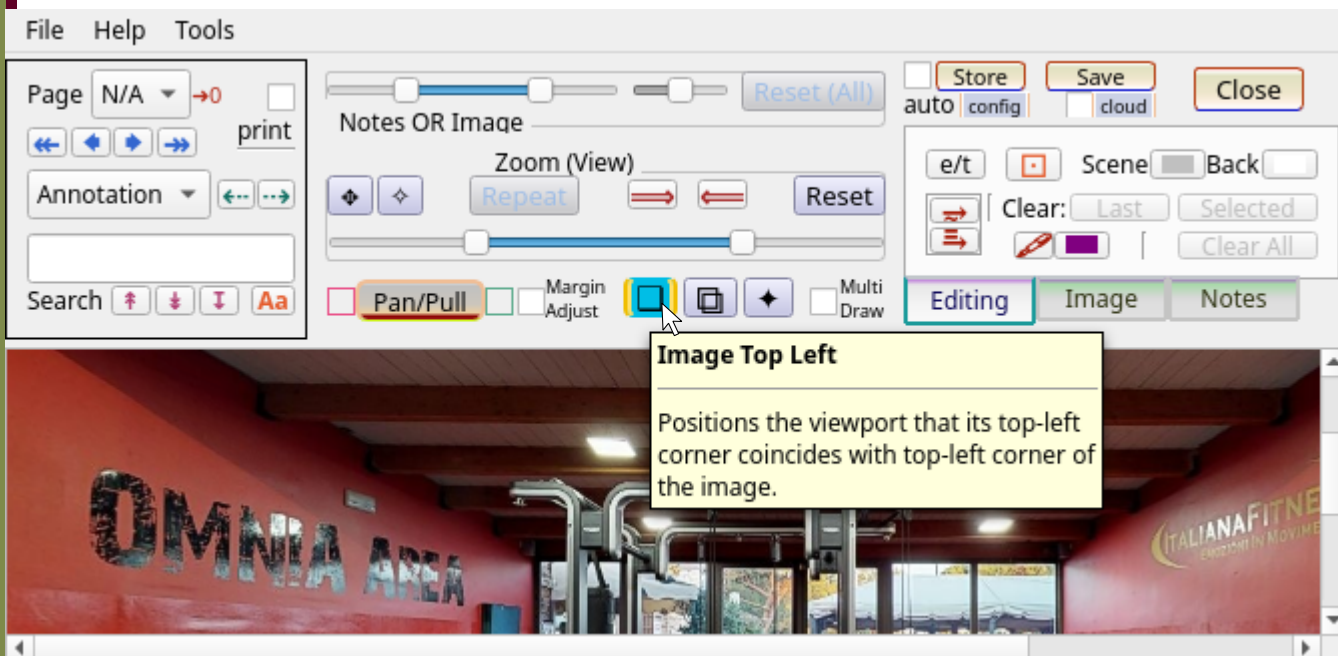
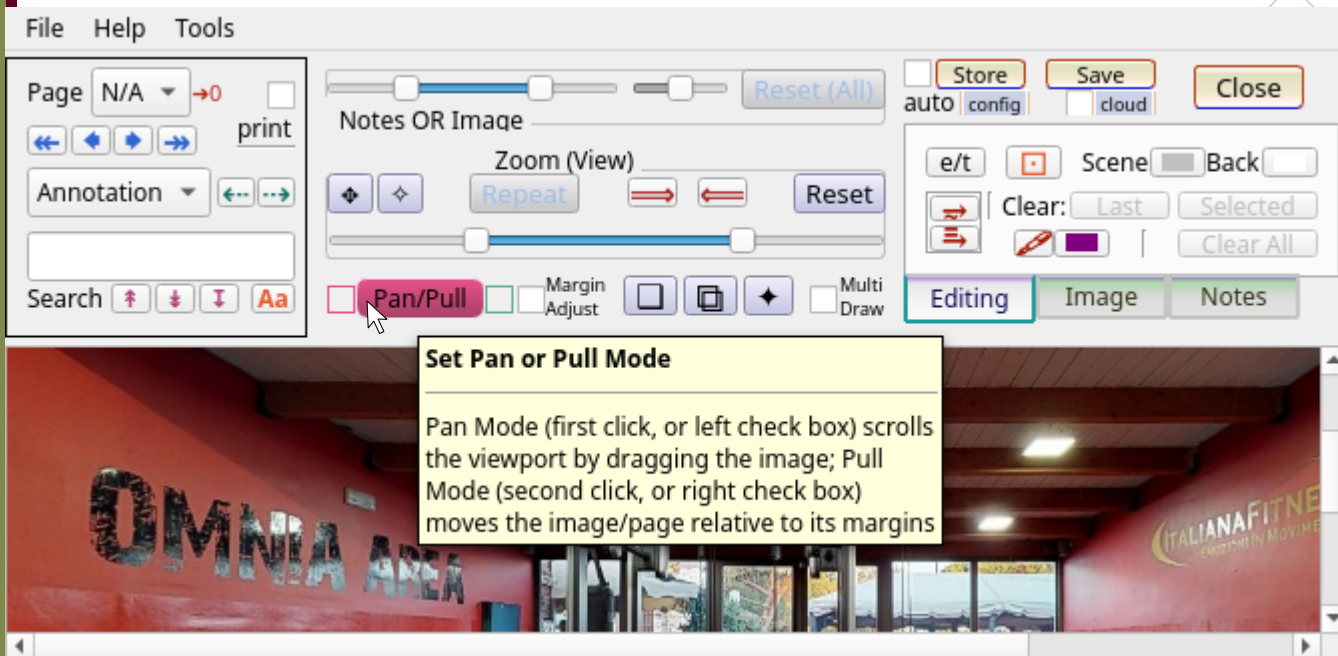
Annotate on PDF (for floor plans, reports, etc.) as well as conventional images (slides 30-32)



Setting up a grid for measurement/reporting annotations



Additional annotation-shape gestures for a measurement/reporting annotation (protruding the top/left mark lines for visual emphasis)



Customizing a presentation framework for displaying images and annotations (slides 33-40)



Page N/A →0 print

Annotation ↩ ↪

Search ↑ ↓ ↕ Aa

Notes OR Image

Zoom (View) Repeat Reset

Pan/Pull Margin Adjust Multi Draw

Store auto config Save cloud Close

e/t Scene Back

Clear: Last Selected Clear All

Editing Image Notes

Center Image

Positions the viewport so that the image appears centered.

Page N/A →0 print

Annotation ↩ ↪

Search ↑ ↓ ↕ Aa

Notes OR Image

Zoom (View) Repeat Reset

Pan/Pull Margin Adjust Multi Draw

Store auto config Save cloud Close

e/t Scene Back

Clear: Last Selected Clear All

Editing Image Notes

Re-Center Image

Moves the image so that it is centered relative to its margins.

Page N/A →0 print

Annotation ↩ ↪

Search ↑ ↓ ↕ Aa

Notes OR Image

Zoom (View) Repeat Reset

Pan/Pull Margin Adjust Multi Draw

Store auto config Save cloud Close

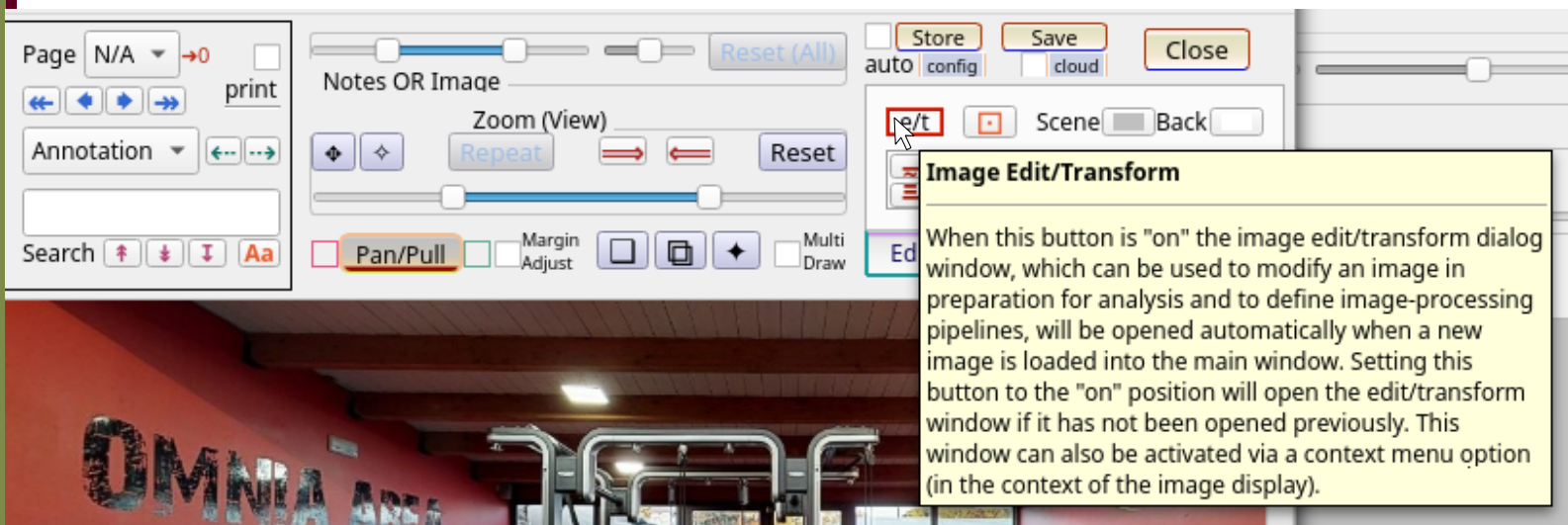
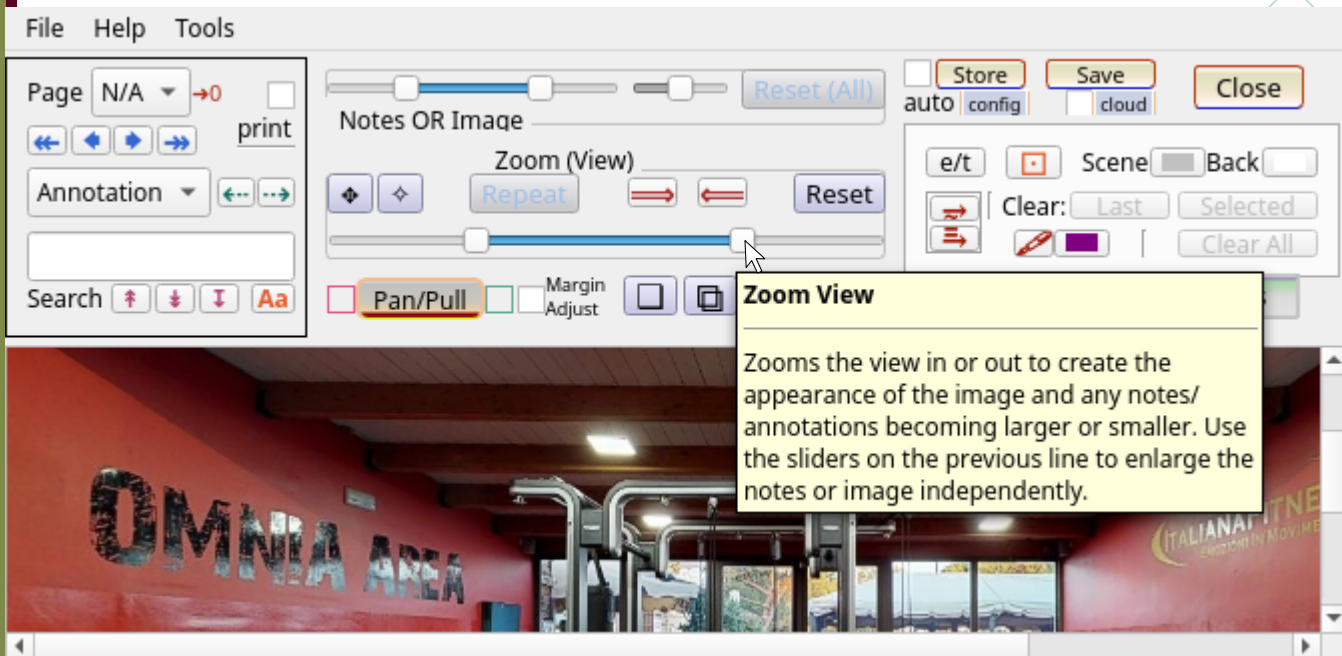
e/t Scene Back

Clear: Last Selected Clear All

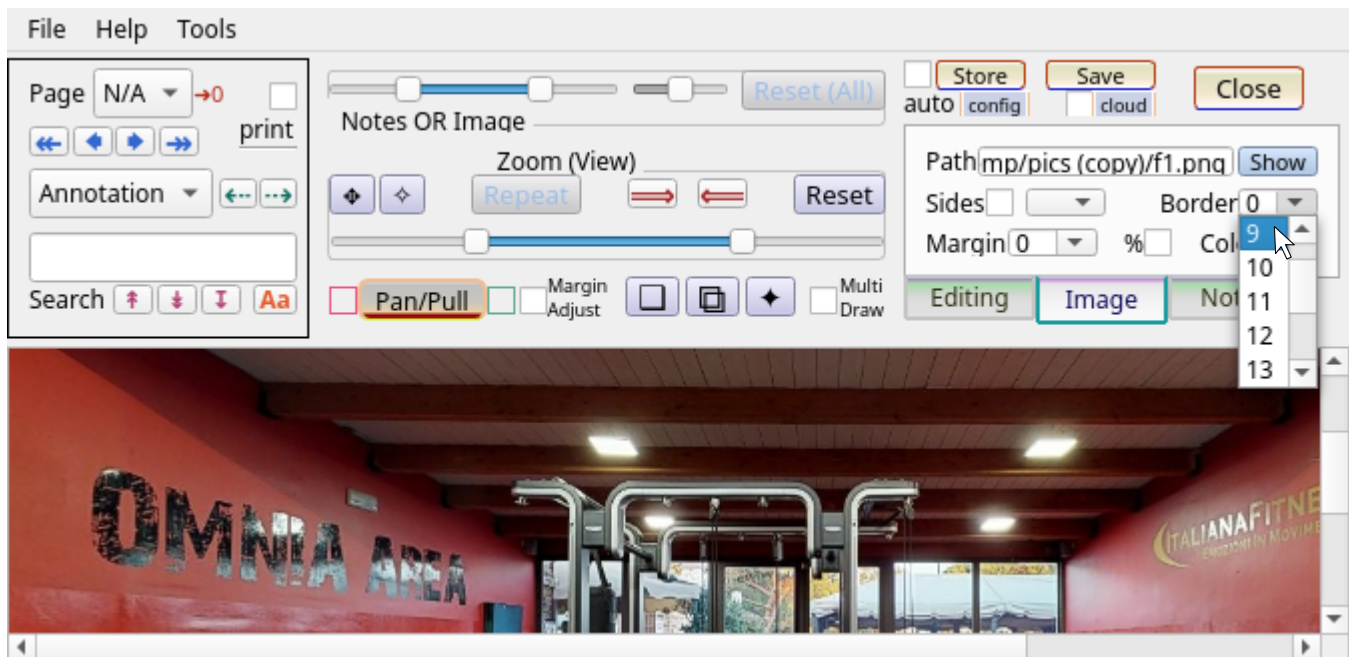
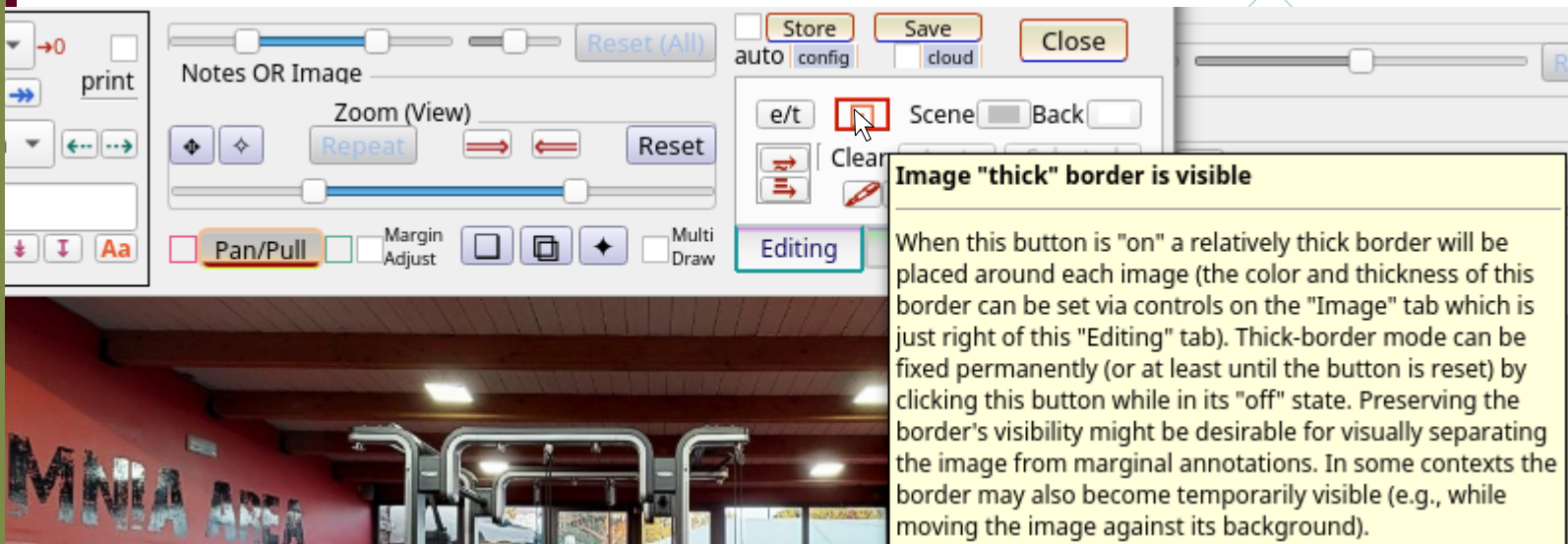
Editing Image Notes

Expand Window

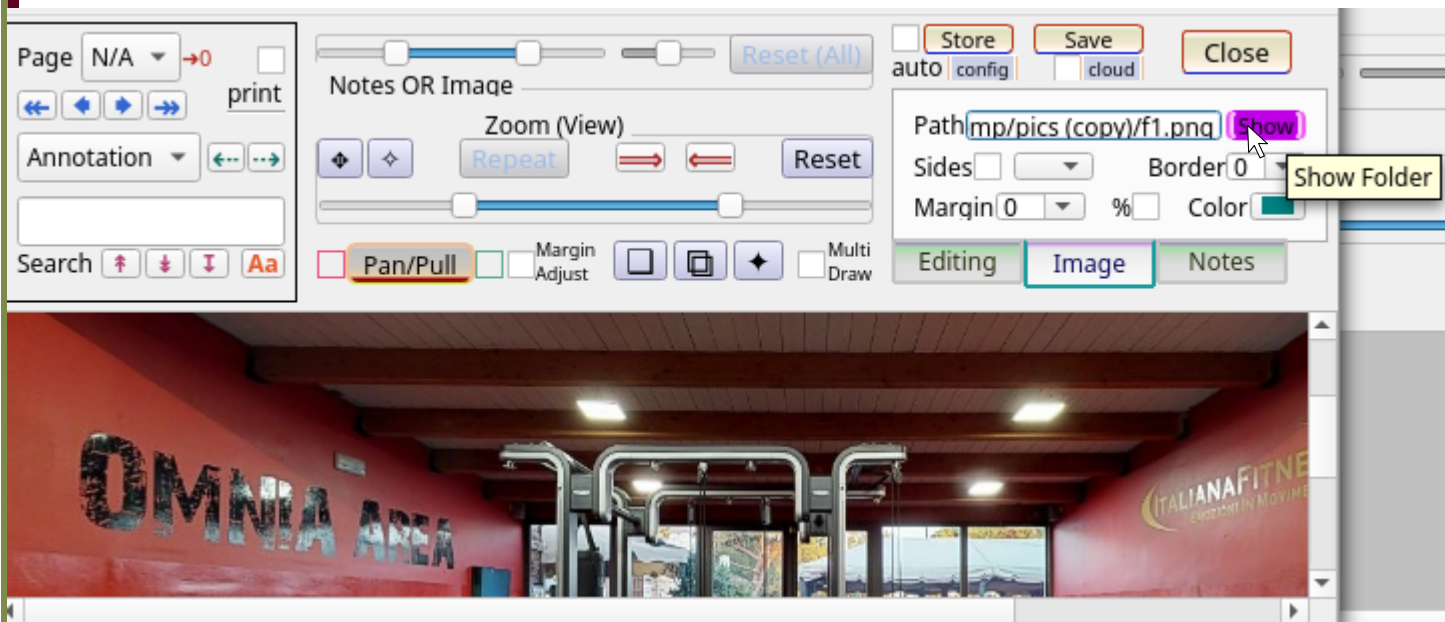
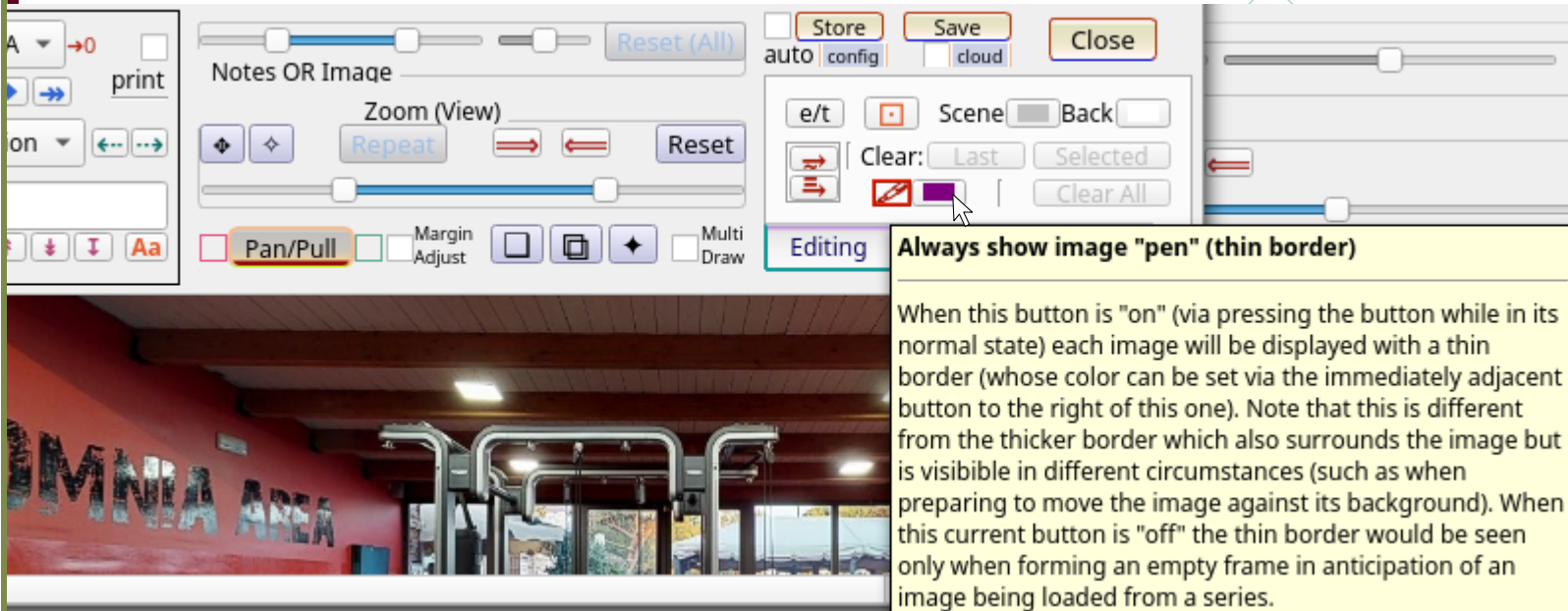
Expands the window to cover the image.



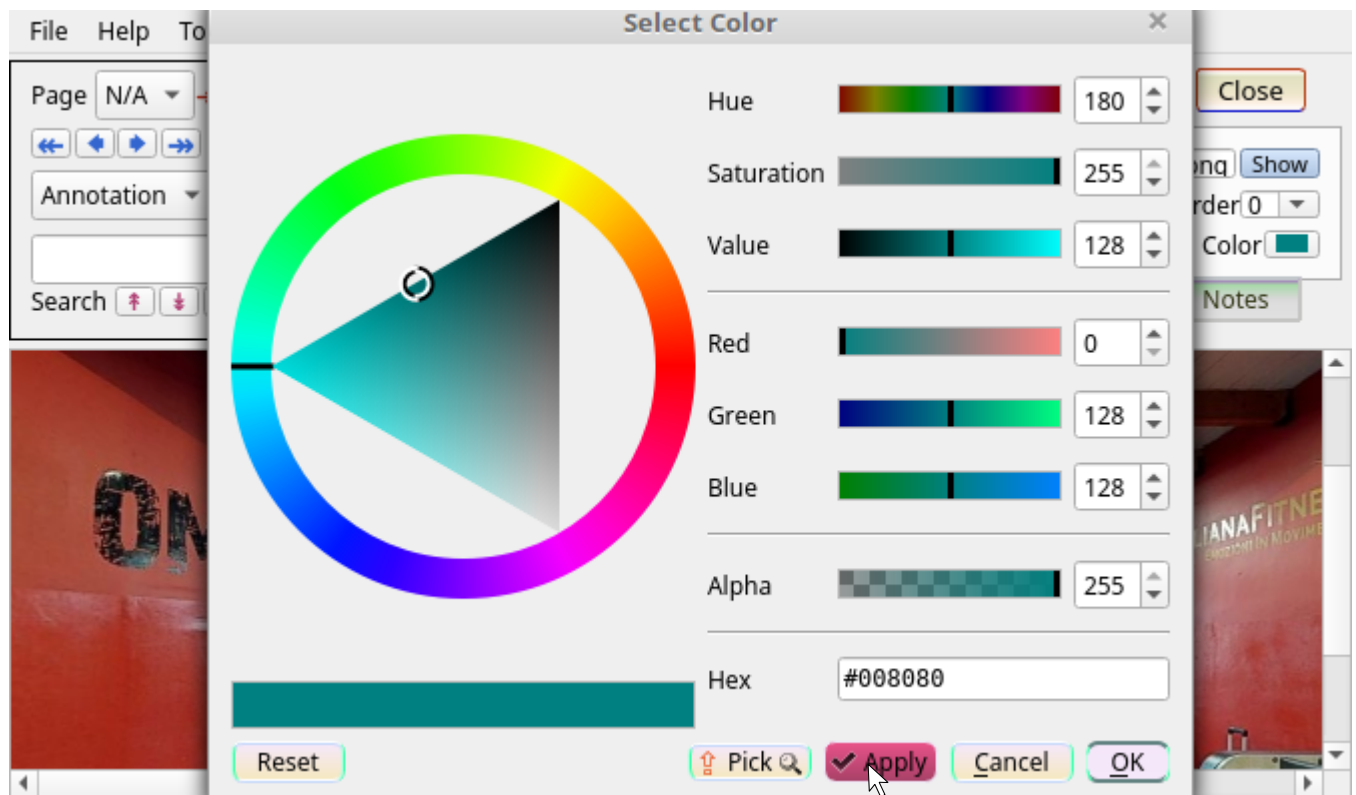
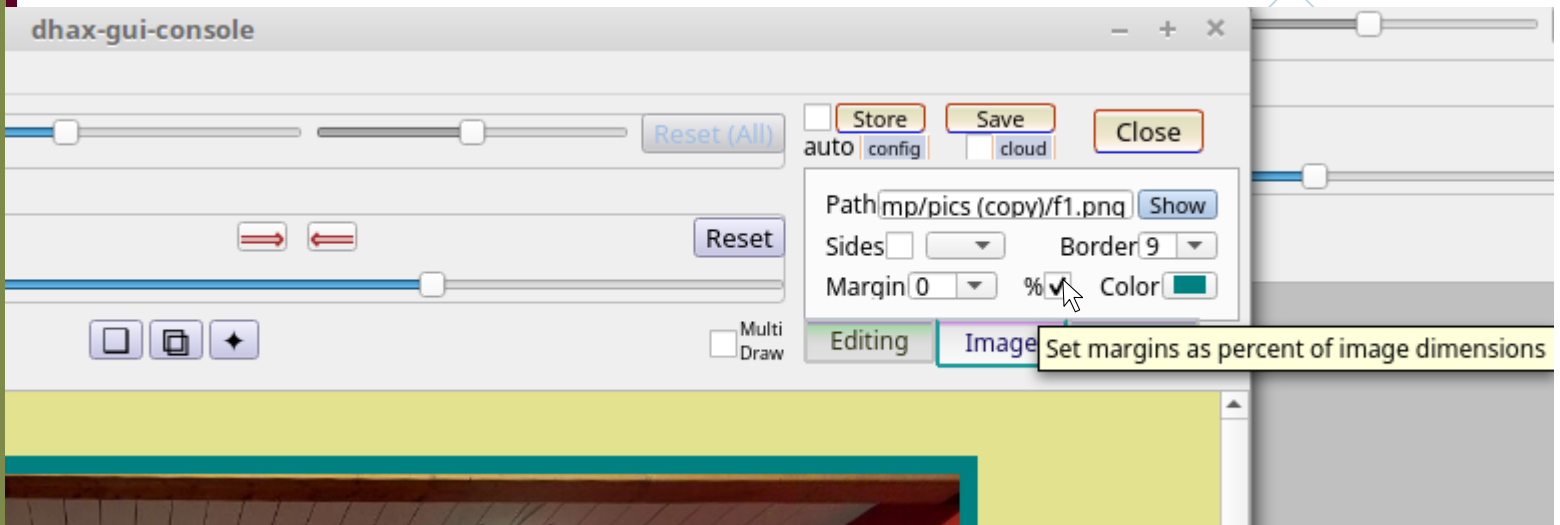
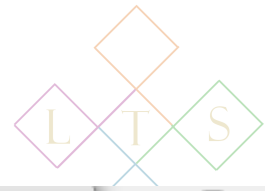
Example GUI controls for setting up image display and workflow features



Styling image borders

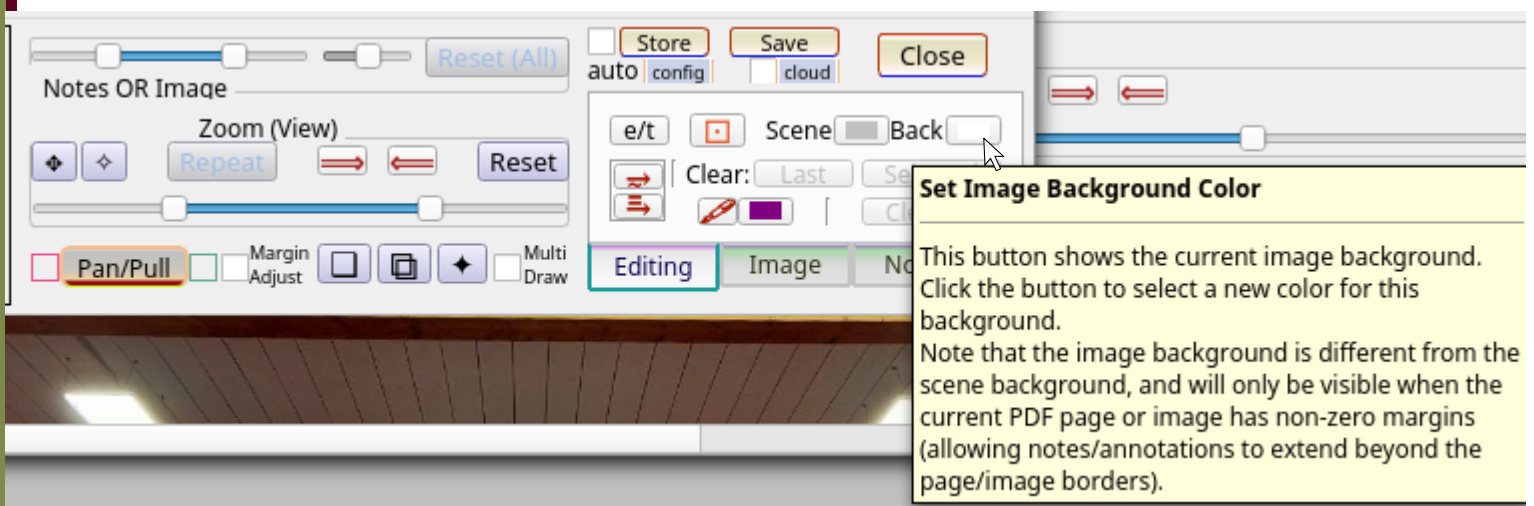
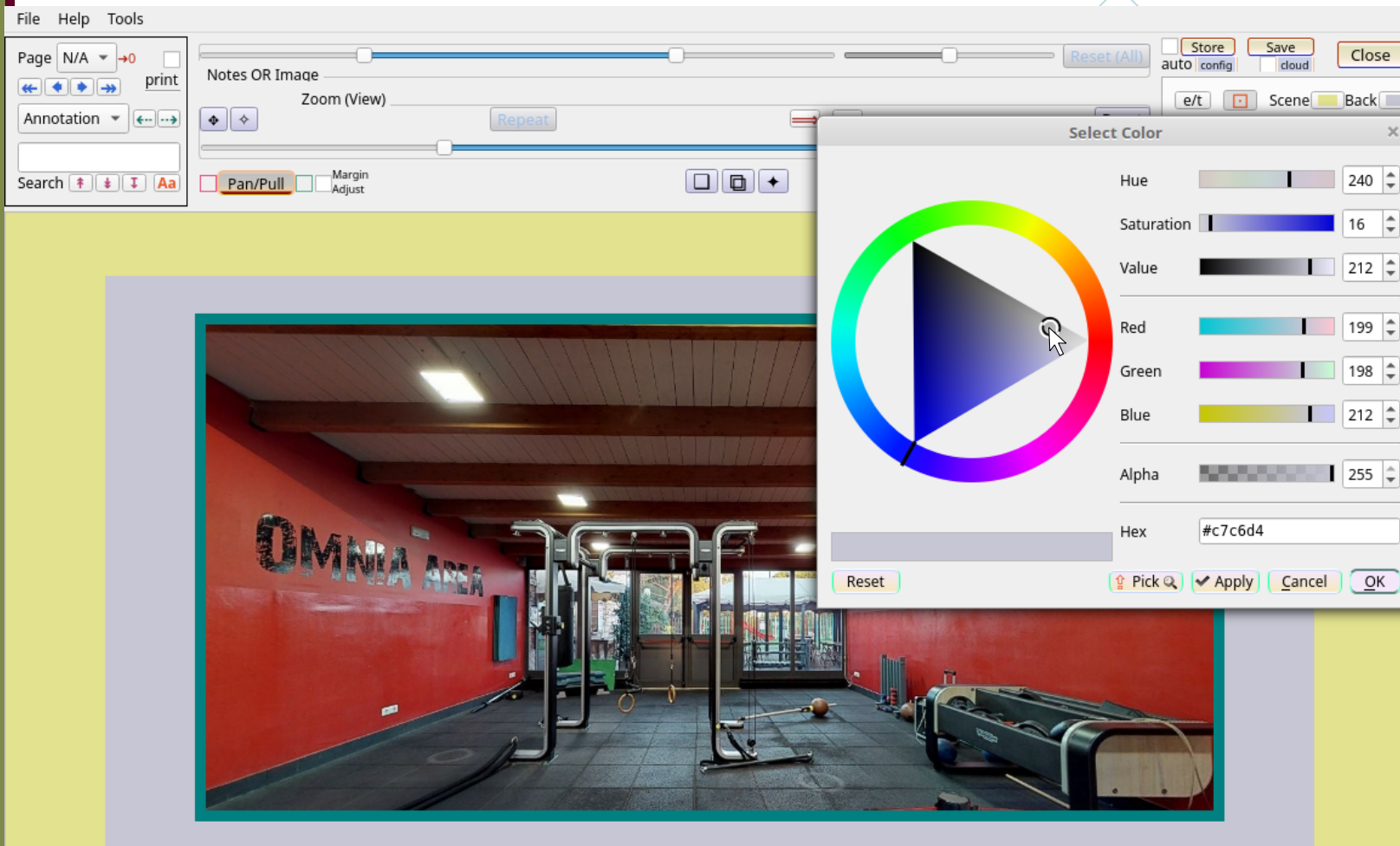
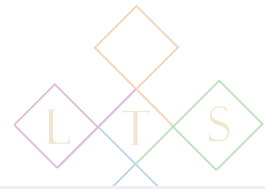


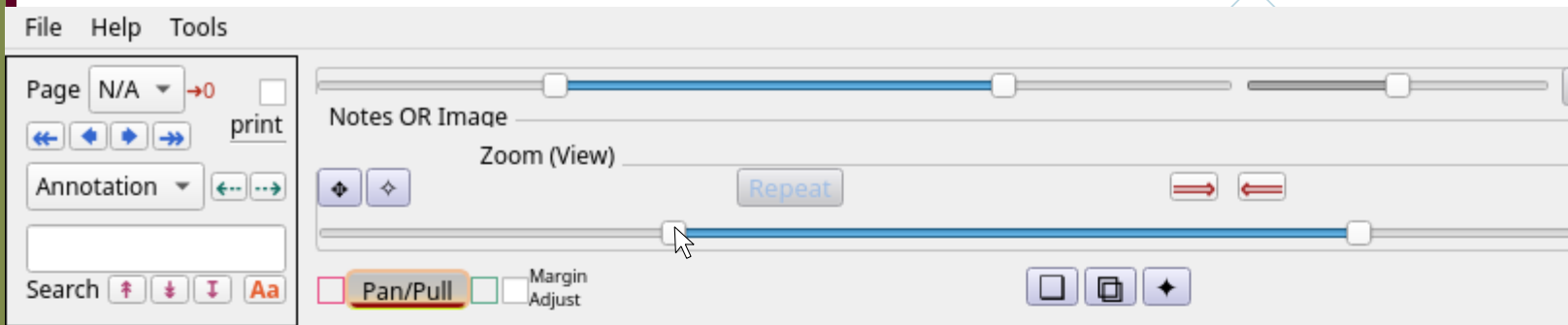
**Browse image series on local filesystem or via a database
(see the bottom screenshot on this slide)**



Presentation metadata includes color and thickness for image margins and background (slides 36, 37 top, and 38-40)

More importantly, the GUI controls and signal-handling framework engineered to define these basic presentation details can be extended for project-specific presentation factors as needed



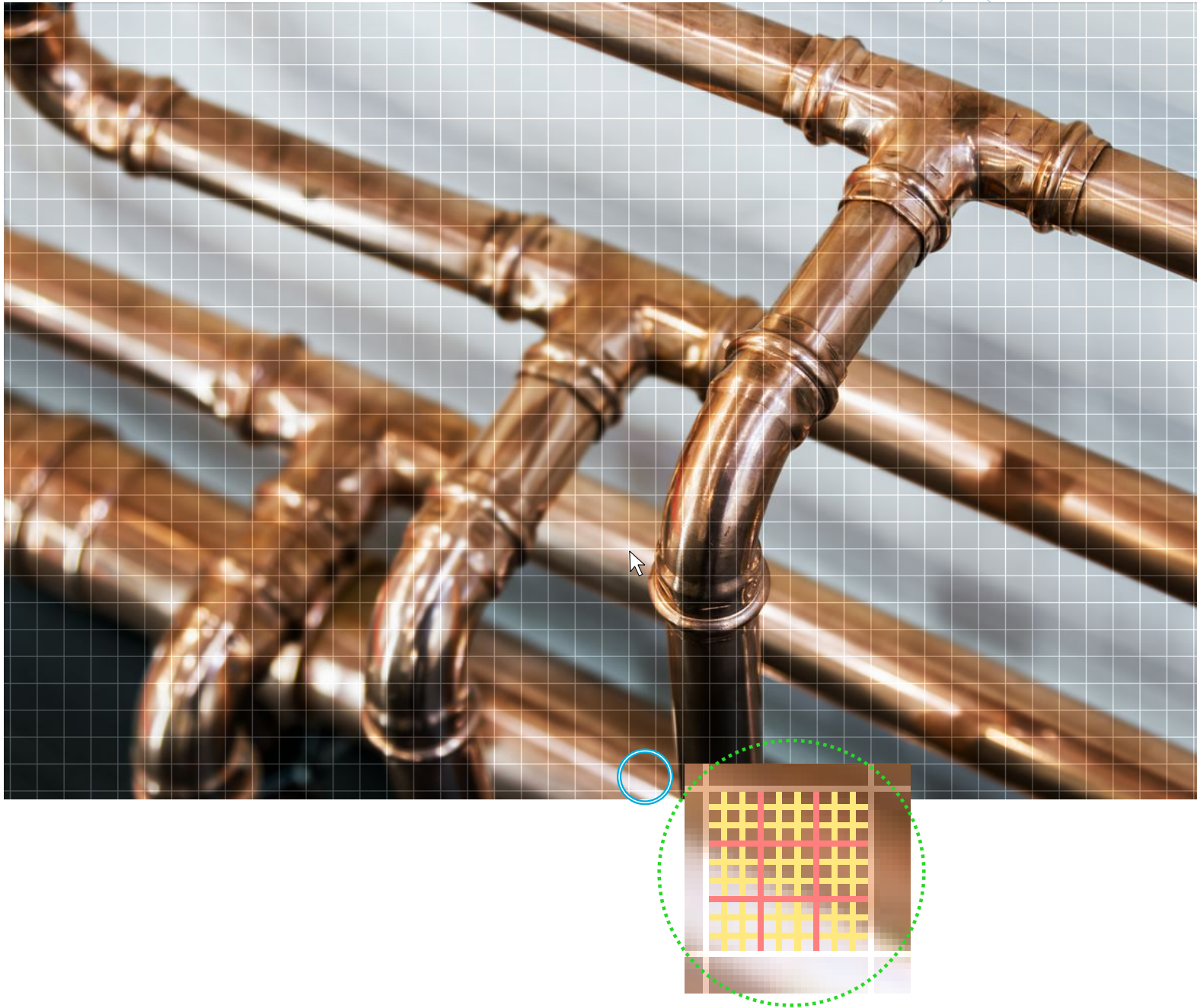
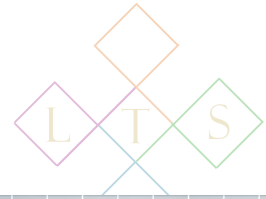


Application view after screen-background color change

Depending on the project, image/scene borders and colors may be treated as annotation/presentation details internal to an image-series or as application-state that can vary between users

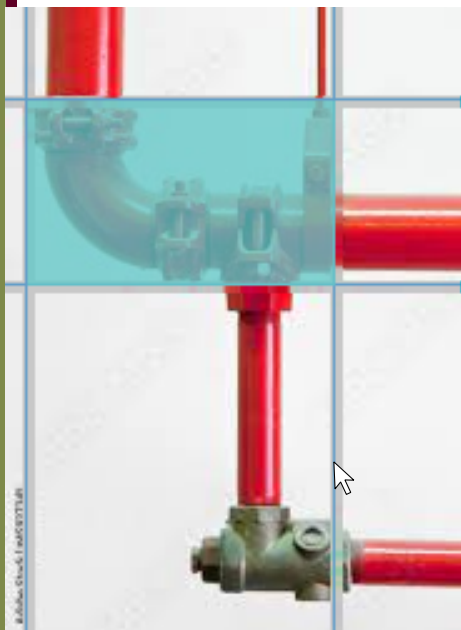
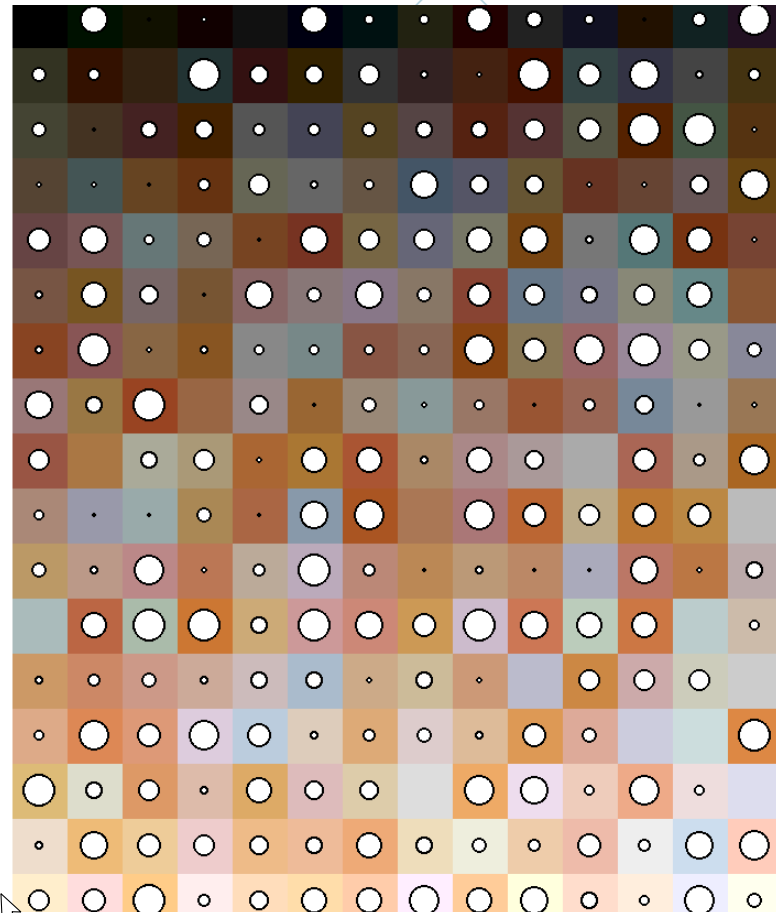
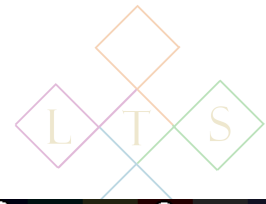
A thickened image border can help isolate the image from the scene background, and can also make it more convenient to drag the image so as to alter its position against the background as desired (using the border to differentiate positional mouse gestures from those, within the image interior, intended for marking annotations)

Image margins can be used for annotations or annotation-text which extends outside the image-boundaries themselves, and in that context margins may need to be expanded by varying amounts for different images



Visualization of XCSD front-tier gridlines, representing the “subdivision” system where XCSD images are grouped into boxes that are further divided in a two- or three-order nested hierarchy (the gridlines are shown via a moderate lightening of the underlying pixels)

For the sake of illustration, one “tier-box” is magnified with lines showing the “mid-tier” and “ground-tier” nonants (using XCSD terms)



Top: One way to leverage special visualization features provided through XCSO; specifically, showing two graphical summaries of the color-frequency distribution of the first pipes image when transformed to use an 8-bit color table

Top-Left view: color blocks are ordered top-left to bottom-right by frequency, “snaking” at the right margin

Top-Right diagram: blocks are ordered darker-to-lighter via HSV value, and the inner-disk size represents frequency in the target image

Left: Another GUI feature internal to the GUI format, specifically, the option to generate mouse-event signals when users hover over gridlines oriented against annotation-shapes



Below are two charts generated by XCS highlighting features of XCS's calculations for assigning pixel memory-layout and organizing boxes around the image-center

3~1 66 0;0	3~0 52 0;0	2~0 45 1;0	1~0 37 1;0	0~0 31 1;0	0~0 32 2;0	1~0 38 2;0	2~0 46 2;0	3~0 53 1;0
56 1+0 2 0 2~1 62 0;0	30 0+0 2 0 2~0 44 0;0	12 0+0 2 0 2~0 26 0;0	12 0+0 2 0 1~0 19 1;0	12 0+0 2 0 0~0 13 1;0	12 0+0 2 0 0~0 14 2;0	12 0+0 2 0 1~0 20 2;0	12 0+0 2 0 2~0 27 1;0	30 0+0 3 0 2~0 47 3;0
56 1+0 1 0 1~1 58 0;0	30 0+0 1 0 1~0 36 0;0	12 0+0 1 0 1~0 18 0;0	2 0+0 1 0 1~0 8 0;0	2 0+0 1 0 0~0 3 1;0	2 0+0 1 0 0~0 4 2;0	2 0+0 1 0 1~0 9 1;0	12 0+0 2 0 1~0 21 3;0	30 0+0 3 0 1~0 39 3;0
56 1+0 0 2 0~1 56 0;0	30 0+0 0 2 0~0 30 0;0	12 0+0 0 2 0~0 12 0;0	2 0+0 1 0 0~0 2 0;0	ath 0 0+0 tierring 0 itr,qc 0 2 psh,mc 0~0 index 0 clk0;0	0 0+0 0 0 0~0 1 1;0	2 0+0 1 0 0~0 5 3;0	12 0+0 2 0 0~0 15 3;0	30 0+0 3 0 0~0 33 3;0
56 1+0 1 2 1~1 61 7;4	30 0+0 1 2 1~0 43 7;0	12 0+0 1 2 1~0 25 7;0	2 0+0 1 0 1~0 11 3;0	2 0+0 1 0 0~0 7 5;0	2 0+0 1 0 0~0 6 4;0	2 0+0 1 0 1~0 10 2;0	12 0+0 2 0 1~0 22 4;0	30 0+0 3 0 1~0 40 4;0
56 1+0 2 2 2~1	30 0+0 2 2 2~0	12 0+0 2 2 2~0	12 0+0 2 2 1~0	12 0+0 2 2 0~0	12 0+0 2 2 0~0	12 0+0 2 2 1~0	12 0+0 2 2 2~0	30 0+0 3 2 2~0

1,2 7	1,3 7	1,4 3	1,5 3	1,6 2	1,7 2	1,8 2	1,9 2
2,6 -2-6 2,2	2,5 -2-5 2,3	2,4 -4-2 2,4	1,3 -3-1 2,5	0,2 -2-0 2,6	1,3 -3-1 2,7	2,4 -4-2 2,8	2,5 -2-5 2,9
1,5 -1-5 3,2	1,4 -1-4 3,3	1,3 -1-3 3,4	1,2 -2-1 3,5	0,1 -1-0 3,6	1,2 -2-1 3,7	1,3 -1-3 3,8	1,4 -1-4 3,9
0,4 0-4 4,2	0,3 0-3 4,3	0,2 0-2 4,4	0,1 0-1 4,5	mask=6 mch=0,0 raw= 0 0 clk 0	0,1 0-1 4,7	0,2 0-2 4,8	0,3 0-3 4,9
0,4 0-4 5,2	0,3 0-3 5,3	0,2 0-2 5,4	0,1 0-1 5,5	0,0 0-0 5,6	0,1 0-1 5,7	0,2 0-2 5,8	0,3 0-3 5,9
1,5 -1-5 23	1,4 -1-4 23	1,3 -1-3 23	1,2 -2-1 23	0,1 1 0 19	1,2 2 1 16	1,3 1 3 15	1,4 1 4 14

XCS ensures that pixels closer to the center are stored in lower memory addresses, so that byte sequences from the start of image data expand outward from the center. Each (canonically 27×27) box corresponds to a contiguous pixel-array offset within the overall image buffer; contiguous pixel runs aligned with box boundaries take the shape of diamonds, octagons, or rectangles around the image center.

Each individual box (in addition to raw pixel data for the portion of the image spanned by its width and height) is associated with data structures recording numerous measurements about its distance and orientation against the image center and its index in the sequence of pixel runs through which XCS encodes images in memory.



Below is a diagram representing calculations of the hybrid Manhattan/Chebyshev distance formulation employed by XCSD:



Each distance in this system is represented by two signed integers corresponding to the diagonal projection of a distance vector and the orthogonal offset of the distance endpoint from the relevant diagonal.

Via this hybrid representation distances can be handled in a strictly integer-based system but with greater specificity than Manhattan or Chebyshev metrics alone (viz., fewer location-pairs are calculated as equidistant), in effect producing a distance definition intermediate between discrete and Euclidean.

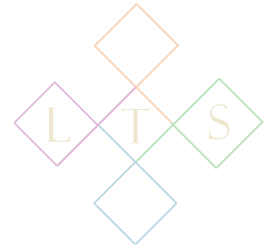




A screenshot indicating mouseover effects for individual (27×27) boxes. The XCSD “tiered” system represents a scaffold through which to define, generate, and respond to user events while interacting with images.



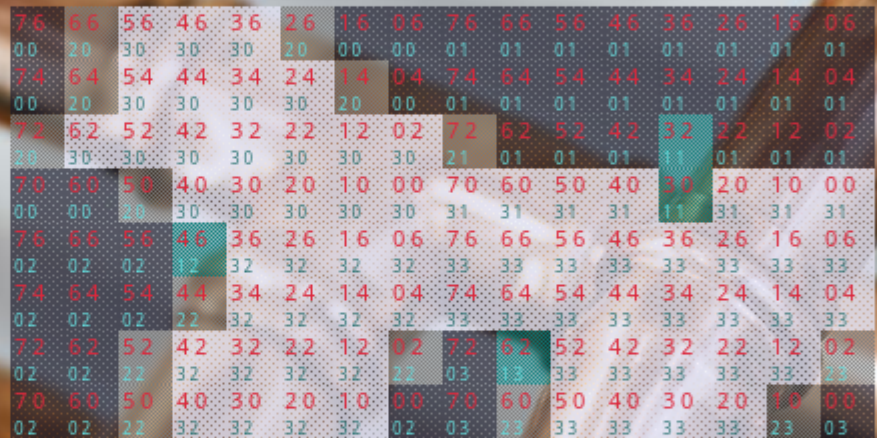
Typically, user events in the context of displayed XCSD images are reported at the level of medium-scale boxes or tiles (called “tierboxes” in XCSD). Mouseover events can be localized to tierbox rows and columns (shown here as yellow and pink highlights) as well as localized to single boxes (as in the yellow-orange intersection). In general, client code responds to event signals which includes a handle to the tierbox where the event is localized.



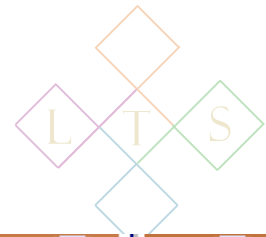
One Form of Cross-Image Navigation

This picture illustrates how XCSD encodes “outer ring” pixels which do not fit into the main image tiling. For these color-values (which might be excluded from some image-processing algorithms), XCSD partitions the outer ring into segments that, in the case of laterally-structured image series, can also be used as a basis for implementing event-signals pertaining to cross-image navigation. Signals such as mouseover styling would be appropriate for series where images are interrelated by diagonal or orthogonal adjacency within some larger graphic from which images are sampled.

Here is a visual summary of a 4-bit-depth segmentation/object-proposal mask: dilating segment boundaries to tiered gridlines (such as 27×27 boxes) allows segments or subregions to be encoded with only a few bytes (including a bounding rectangle and a box-by-box mask) as opposed to at-scale binary co-images, enabling real-time user-signals localized to Regions of Interest as well as direct in-database processing of object proposals.



While simple foreground/background binaries can be useful, this example shows a 4-valued bitmask (foreground in near-white, background in near-black) where intermediate values (in grey) represent boxes straddling foreground/background borders. A fourth option (shown here in teal/bluegreen) represents non-standard border segments (such as two discontinuous border fragments) or, alternatively, might be used as a flag marking that the associated box has an additional, more detailed data structure defined as an extension.



Below is a demonstration of XCSO functionality aiding image-analysis code, showing a high-zoom rendering of a single 27×27 box where each pixel can be examined by RGBA and HSV colors as well as in-box coordinates:



Red and purple lines represent 9×9 and 3×3 pixel groups, and the 3×3 group centers are marked with grey background. Alpha channels use XCSO's convention wherein lower numbers correspond to greater opacity. For images without varying transparency, alpha channel values can be replaced by colors, indices or bitmasks representing "extension" channels (including non-chromatic).

Developers can generate diagrams similar to this one (using built-in XCSO functions) if it is necessary to examine images on a pixel-by-pixel basis while constructing or testing processing-algorithms.